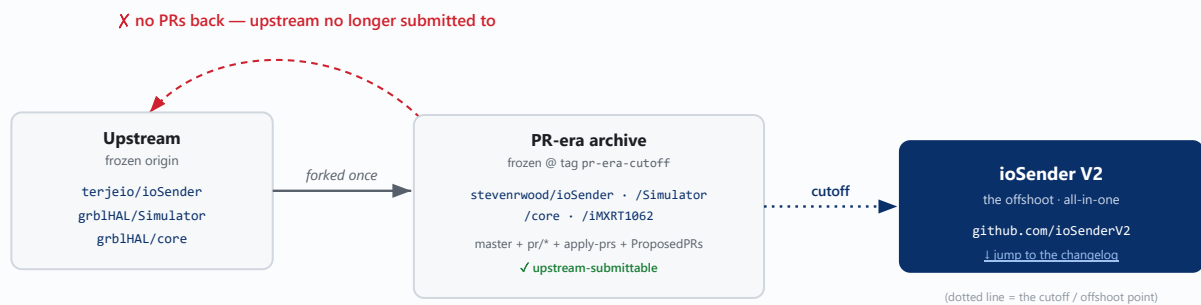


ioSender V2 — Overview

An enhanced **all-in-one** grblHAL / Grbl g-code sender. ioSender V2 is the ongoing offshoot of [stevenrwood/ioSender](#) (itself a one-time fork of [terjeio/ioSender](#)), severed from upstream at the **PR-era cutoff**. There are no composable feature branches and nothing is submitted upstream any more — the product is the `master` branch, consumed whole. This page maps how V2 came to be and how its repos wire together at runtime; the [full changelog](#) follows below.



One fork from each upstream → the PR-era archive (everything up to the cutoff, still submittable) → a dotted hand-off across the cutoff to **ioSender V2**, where all development now happens. Upstream is a frozen origin with the return path severed.

V2 repos & how they wire together at runtime

The app and simulator moved to the **ioSenderV2** org (original names kept, full git history preserved). The grblHAL **firmware** was *not* migrated — its deep submodule web makes it all-or-nothing, so it stays in the frozen fork, referenced by the `pr-era-cutoff` tag.

ioSenderV2/ioSender

The WPF sender app (this document lives here). Talks to the controller over USB serial or Telnet/WebSocket.

ioSenderV2/Simulator

Option-matched simulator. On connect the app reads the controller's build options → a 12-hex signature, then pulls a matching sim from this repo's `sim-<sig>` releases (a shared build cache), dispatching the `build-matched-sim` CI to produce one if absent. Every download is public; a token is only needed to *trigger* a build.

stevenrwood/IMXRT1062

firmware · tag-only

The grblHAL firmware build (Teensy 4.1 driver superproject + `core` submodule at `srw/combined`, incl. the WCS-rotation transform fix) the app connects to. Stays in the fork — build branch `srw/local-build-config`, frozen reference at tag `pr-era-cutoff`.

Features & Fixes

Fork `stevenrwood/ioSender` · base branch `master` · **55 improvements in the integration build**. The fork ships as one all-in-one build — every change below is already in the `integration` branch and consumed wholesale. · all LF-normalized (`.gitattributes * text=auto eol=lf`).

Features at a glance — grouped by area. **NEW** new capability, **CHG** changes existing behaviour, **FIX** bug fix. Numbers link to the detail below.

Connectivity & reliability

Connect to USB / Ethernet / simulator, with a Connect/Reconnect menu, remembered host and opt-in prefer-network	NEW	15
Validate controller — streams a capability-tailored test in check mode (no motion), reports pass/fail + 3D preview	NEW	16
Restore main-window <i>position</i> across launches (was size only)	CHG	17
Survive connection loss / SD-card swap — guarded writes + auto-reconnect instead of crashing	FIX	4
Telnet / websocket modal-dialog UI deadlock	FIX	24

Jogging & main page

Configurable main page — Edit Main Page editor, pinnable flyouts, assignable jog panels, per-offset Go/Set buttons	NEW	9
Native Xbox / game-controller support — analog-stick jog, remappable buttons, live Controller tab	NEW	8
3D view: Ctrl+click to jog (retracts Z first) + per-axis \$23 orientation fix	NEW	20
Keyboard jog keeps working when focus drifts off the Job view	CHG	22
High-DPI / 125–150% text-scaling layout	CHG	7
Assignable <i>Jog distance</i> +/- controller actions — step the jog distance preset (separate from jog speed)	NEW	35
Run-bar jog preset spinners — read-only <i>Dist</i> / <i>Feed</i> up/down selectors (wrap-around) on the Check line; startup splash stays on top so the full init sequence is visible	NEW	45

Machine setup & settings

Machine Setup Wizard — guided first-run machine description (catalog, home-corner picker, per-axis table)	NEW	21
grbl:Settings — change-from-startup highlight, granular revert, per-Save restore points, bitfield tooltips	NEW	18
Stepper calibration (scratch) tab — long-span V-bit steps/mm	NEW	5
Safe-Z on Go To — lift / traverse / descend instead of a diagonal rapid	CHG	23
Editable G28/G30 predefined offsets (Set rapids to target, then stores)	CHG	26
Machine Setup restructure — Settings split (Grbl/App), new Tools tab, guided step-tabs + startup gate	NEW	28
Settings redesign — flat tab strip, free-text \$-setting search (with match nav), grbl-settings autosave, dialog editors folded in as tabs	CHG	42

Per-user config folder + standalone <code>.xml</code> folded into <code>App.config</code> sections (<code>ConfigPanel<T></code> wizards, curated defaults); console output search + font size, shared reset footer, bindable <code>ResetAndUnlock</code>	CHG	43
Lathe mode toggle — App > Main checkbox enables/disables lathe mode, adding or removing the Lathe wizards tab (restart to apply)	NEW	47
Restart-needed cue — the Restart button pulses red while a restart-only change is pending and offers to relaunch when you leave Settings	CHG	48

Commissioning & workflow tools

Surface Spoilboard — machine-referenced facing (Z-only touch-off, Reuse Z0, finish pass, leveling check)	NEW	29
Load Stock — corner-probe stock size/flatness/skew/thickness, results popup, Copy size → Fusion add-in	NEW	30
Squareness (Auto Square) — drill an L, measure, write the ganged-axis squaring offset + re-home	NEW	31
Probe definition diagrams — per-type schematic in the probe editor labelling the measurements to enter	NEW	34
Probe input follows the selected probe type — tool setter → Q1, else the main probe → Q0 (guards a stale selection)	FIX	36
Start Job (was Load Stock) — first tab, connection-gated tabs, auto 3D-probe at the front-left origin, writes <code>start_job_macro</code> ; new Verify skew pass (re-touch each corner in the rotated frame); flyout persistence & offset-Set fixes	NEW	44
Tools tab guidance — each Tools tab (tool table, spoilboard surfacing, scratch calibration, Trinamic, PID) now shows an on-screen “About” explainer of what it does and how to use it	NEW	52

Files, SD card & ATC

Load Folder — assemble per-toolpath <code>.nc</code> files into one outlined program (+ Fusion add-in)	NEW	6
SD + LittleFS browser & file manager — multi-filesystem, View/Edit/Create/Upload via Notepad	NEW	14
SD Card first-line cleanup — the filesystem status no longer overlaps the checkbox, and the free-space banner is hidden when the controller reports no sizes (was a bare “mounted”)	FIX	51
ATC tool-change macro provisioning — Install ATC macros + seeded Start Job	NEW	25

Program view & G-code

Reusable program view — file / folder / each wizard gets its own view; one Cycle Start runs the active one	CHG	32
Background file / folder loading — big programs parse on a worker thread; the UI stays responsive, rows fill as read	CHG	32
Continuous block numbering — explicit N words shown in the Block column and hidden from the G-code column	CHG	32
Explain G-code on hover — plain-language per line/selection (decodes parameters, O-word CALLs, expressions); click the balloon to copy	NEW	33

One in-place streaming path — every wizard/macro runs in its own view; a run never overwrites the loaded job	CHG	37
Live 3D carve view — watch the stock get machined away (program-sized stock, real cutter profile flat/ball/V-bit, Play / Cycle Start playback) instead of just toolpath lines	NEW	41
Macros, console & keys		
Macro manager dialog (grid, F-key assign, confirm-on-run flag) + run-only macro flyout	NEW	10
Macro pre-processor directives — PREREQ / MBOX / WAITIDLE / PROMPT / @path	NEW	11
In-app Key Mappings editor + console state persistence	NEW	3
Console: inline terminal command prompt + output right-click menu (Clear / Save)	NEW	19
Polish, onboarding & docs		
High-DPI polish — fixed pixel sizes converted to Min/Auto so text doesn't clip at 125–150% scaling	FIX	46
Lathe wizards rebuilt to the app layout idiom — the 5 upstream absolute-Margin wizards re-hosted in scaling StackPanel/Grid so they no longer clip/overlap at 125–150%	CHG	49
Tab strips fill their width — a StretchTabControl spreads each tab header proportionally across the row so tabs read as distinct (main bar + Settings / Tools / Lathe / Probing / Machine Setup)	CHG	50
Onboarding tooltips — plain-language controller-state field + per-setting help	NEW	12
UI localization — all UI strings registered for 7 locales (cross-cutting)	NEW	9, 10, 12, 14–16, 18, 19, 21, 23
Center owner-less modal dialogs (MPGPending, ProbeVerify)	FIX	27
Show Motor Fault / Warning signals in the Signals display	FIX	2
Build documentation — Mega-XL upgrade notes + repeatable tool-change guide	NEW	13
Assorted released-ioSender bug fixes (RP.Math build, LF endings, CSV null-ref, boolean settings, null-comms)	FIX	1
Headless build — build.ps1 + VS Code tasks build/launch without opening the VS GUI (MSBuild found via vswhere); unhandled exceptions now dump to %AppData%\ioSender\ioSender.crash.log and exit 0xFA11 instead of blocking on a dialog	NEW	53
Code renamed LoadStock → Start Job to match the UI throughout (files, classes, ViewType, layout keys, config section)	CHG	54
Localization catch-up — later V2 features (Start Job, probing-redesign dialogs, main-page editor, Tools guidance, ...) were x:Uid'd but had no LocBaml rows; every missing string backfilled into all 7 locale CSVs	NEW	55

1	Bug Fixes for released ioSender	Files: 5 0 0 Lines: +28 -10
2	Surface Motor Fault / Warning signals	Files: 2 0 0 Lines: +5 -32
3	Console persistence + Key Mappings editor	Files: 12 0 +4 Lines: +1224 -15
4	Connection-loss handling + auto-reconnect	Files: 10 0 0 Lines: +522 -63
5	Stepper calibration (scratch) tab	Files: 13 0 +3 Lines: +773 -2
6	Load Folder + Fusion batch-post add-in	Files: 28 0 +8 Lines: +1634 -8
7	High-DPI / large-text layout	Files: 27 0 0 Lines: +126 -109
8	Xbox controller + Key Mappings editor polish	Files: 14 0 +4 Lines: +2255 -687
9	Configurable main page + flyouts + jog panels (XL)	Files: 31 0 +20 Lines: +2835 -119
10	Macro manager + run-only flyout	Files: 16 0 +2 Lines: +598 -26
11	Macro Processor Enhancements	Files: 8 0 +1 Lines: +723 -21
12	Onboarding tooltips	Files: 14 0 0 Lines: +283 -28
13	Build documentation (Mega-XL notes + tool-change guide)	Files: 1 0 +8 Lines: +2662 0
14	SD + LittleFS file browser & file manager	Files: 12 0 +3 Lines: +945 -135
15	USB / Ethernet / simulator connection support + Connect menu	Files: 26 0 +3 Lines: +1151 -68
16	Validate controller (check-mode g-code conformance test)	Files: 19 0 +1 Lines: +1208 -38
17	Restore main window position across launches	Files: 2 0 0 Lines: +60 -7
18	grbl:Settings — live edits, change highlight, revert & snapshots	Files: 12 0 +2 Lines: +659 -43
19	Console — inline terminal-style command prompt	Files: 13 0 0 Lines: +220 -21
20	3D view — Ctrl+click to jog + per-axis orientation	Files: 5 0 0 Lines: +206 -60
21	grbl:Settings — Machine Setup Wizard	Files: 12 0 +3 Lines: +1740 -1

22	Keyboard jog — keep active when Job-view focus drifts	Files: 2 0 0 Lines: +23 -1
23	Go To — optional safe-Z (lift, traverse, descend)	Files: 10 0 0 Lines: +54 -8
24	comms — fix telnet/websocket UI deadlock	Files: 2 0 0 Lines: +29 -10
25	ATC tool-change macro provisioning (Install ATC + Start Job)	Files: 7 0 +5 Lines: +804 -10
26	Offsets — editable G28/G30 (Set rapids & stores)	Files: 1 0 0 Lines: +21 -2
27	Dialogs — center owner-less modal dialogs	Files: 4 0 0 Lines: +4 -2
28	Machine Setup restructure + Tools tab + setup gate	Files: 4 0 +2 Lines: +237 -21
29	Surface Spoilboard — machine-referenced facing tool	Files: 2 0 +2 Lines: +673 0
30	Load Stock — corner-probe stock measurement	Files: 7 0 +7 Lines: +1125 0
31	Squareness (Auto Square) — ganged-gantry offset tool	Files: 2 0 +2 Lines: +686 0
32	Program view as a reusable object + background loading	Files: 20 0 +4 Lines: +830 -184
33	Explain G-code on hover (novice tooltips)	Files: 4 0 +1 Lines: +805 -39
34	Probe definition diagrams	Files: 2 0 0 Lines: +142 -22
35	Jog distance +/- controller actions	Files: 4 0 0 Lines: +17 -2
36	Probe input follows the selected probe type	Files: 3 0 0 Lines: +16 0
37	One in-place streaming path (stayPut removed)	Files: 3 0 0 Lines: +62 -110
38	Match the bundled simulator to the connected controller	Files: 2 0 0 Lines: +355 0
39	Re-establish modal state on a mid-program start	Files: 1 0 0 Lines: +22 0
40	Load Stock “apply work rotation” defaults off	Files: 1 0 0 Lines: +6 -1
41	Live 3D toolpath + material-removal carve view	Files: 4 0 +2 Lines: +1107 -62
42	Settings redesign — flat tab strip + searchable Grbl settings	Files: 23 2 0 Lines: +935 -542

43	Per-user config folder + standalone config folded into App.config	Files: 27 0 +3 Lines: +1269 -369
44	Start Job workflow + Verify skew, flyout & offset fixes	Files: 17 0 0 Lines: +369 -182
45	Run-bar jog preset spinners + startup splash + MDI Send align	Files: 4 0 +2 Lines: +132 -9
46	High-DPI audit — text no longer clips at 125–150% scaling	Files: 63 0 0 Lines: +196 -197
47	Lathe mode enable/disable checkbox	Files: 3 0 0 Lines: +56 -0
48	Restart button pulse + restart-on-leave prompt	Files: 2 0 0 Lines: +58 -1
49	Lathe wizards rebuilt to the app layout idiom (high-DPI)	Files: 5 0 0 Lines: +461 -387
50	Tab strips fill their width (StretchTabControl)	Files: 7 0 +1 Lines: +137 -14
51	SD Card first-line cleanup (overlap + free-space banner)	Files: 2 0 0 Lines: +9 -4
52	Per-tool guidance text on the Tools tabs	Files: 7 0 0 Lines: +46 -14
53	Headless build script + crash logging	Files: 2 0 +2 Lines: +255 -8
54	Rename LoadStock → Start Job (code matches the UI)	Files: 30 0 0 Lines: +177 -177
55	Localization sweep — V2 strings into all 7 locales	Files: 29 0 0 Lines: +1581 -1
Totals (55 changes)		Files: 476 2 +92 Lines: +30693 -3872

Cross-repo: which ioSender features the stevenrwood/Simulator fork exercises.

No build dependency, and all-or-nothing — you run the whole Simulator@integration build against ioSender (not a chosen subset), so this table is informational rather than something to satisfy. The coupling is runtime / protocol: the simulator only exercises a feature when the ioSender change below drives it over the socket.

ioSender feature	Simulator feature it lights up	Coupling
#6 Load Folder	3D carve view	PushHeaderToSimulator() sends the program's leading (STOCK ...) / (TOOL ...) comments so the carve knows stock size + cutter geometry.
#11 macro pre-processor	littlefs macros	O<name> CALL forwarding resolves macros stored on the simulator's /littlefs.
#14 SD / filesystem	littlefs, YModem	File browser + YModem uploader that puts files onto the simulator filesystem.

browser

#15 connect-to-simulator	3D carve view, settings persistence, sim core	Connects to the standalone sim; <i>always-send-comments</i> forces (TOOL)/(STOCK) through for the carve; "My Machine" EEPROM uses the sim's settings persistence.
#25 ATC macro provisioning	YModem, littlefs, probe surfaces	"Install ATC macros" uploads via YModem to /littlefs; the Start-Job / <code>probe_tfl</code> cycle drives the modelled probe surfaces.

For the full virtual-CNC experience, run **ioSender@integration** against **Simulator@integration** — one all-in-one bundle each (the firmware-submodule fixes ride along inside the sim). The matched-simulator feature ([#38](#)) builds an option-matched sim automatically.

#1 Bug Fixes for released ioSender

File	+	-
.gitattributes	3	2
CNC Controls/CNC Controls/JobControl.xaml.cs	7	0
CNC Controls/CNC Controls/Widget.cs	3	1
CNC Core/CNC Core/CNC Core.csproj	6	0
CNC Core/CNC Core/Grbl.cs	9	7

A bucket of fixes for bugs that exist in released ioSender, independent of the feature PRs. Each is small and self-contained:

- **RP.Math build fix.** CNC Core compiles `GCodeEmulator.cs`, which uses the `RP.Math.Vector3` type, but the project file declared no reference to `RP.Math`, so a clean build fails to resolve the type. Adds the `RP.Math` and `RP.Math.Vector3` references using the *same* `HintPath` convention already used by `CNC Controls.csproj` (`..\..\Vector-master\RP.Math.Vector3\bin\Release\`). Purely additive.
- **LF line-ending guard.** Adds `.gitattributes (* text=auto eol=lf)` so line endings are normalized.
- **CSV null-reference fix.** Avoids a `NullReferenceException` when an alarm/error/settings-group CSV resource is missing.
- **Boolean settings convention.** The settings editor checkbox initialised with `value == "1"`, but a boolean is true when `value != "0"` everywhere else (the list, controller-value parsing). Booleans reported as a truthy value other than the literal "1" showed enabled in the list yet left the checkbox unchecked. Canonicalises boolean values to "1"/"0" in `GrblSettingDetails.Value` and reads the checkbox with the `!= "0"` convention.
- **Null-comms streaming guard.** `JobControl.ResponseReceived` is raised by a specific comms instance but writes to the static `Comms.com`; during a reconnect/teardown (startup handshake or relaunch) that static can be null while an in-flight response from the old link still arrives, NREing in the `SendMDI/Reset` cases. Bails out early when `Comms.com` is null.

#2 Surface Motor Fault and Motor Warning signals in Signals display

File	+	-
CNC Controls/CNC Controls/SignalsControl.xaml	2	30

Removes the capability-gating Styles that prevented the Motor Fault (**F**) and Motor Warning (**W**) indicators from ever showing.

grblHAL does not advertise these as NEWOPT capability tokens, so the visibility gates were effectively dead code — the LEDs were permanently hidden. Motor Fault / Warning are now always displayed alongside the other signal LEDs (H / S / P), turning red when the corresponding signal triggers. This surfaces the Motor Fault signal as intended, without requiring the firmware to advertise a capability flag.

#3 Persist console state and add an in-app Key Mappings editor

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	14	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	16	3
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/ConsoleControl.xaml	3	3
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	14	0
CNC Controls/CNC Controls/ConsoleWindow.xaml.cs	47	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	115	0
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	606	0
CNC Core/CNC Core/CNC Core.csproj	7	0
CNC Core/CNC Core/KeyPressHandler.cs	111	0
CNC Core/CNC Core/ShortcutKey.cs	136	0
ioSender XL/ioSender XL/App.config	6	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	69	3
ioSender/ioSender/MainWindow.xaml.cs	69	3

Persists the console's state across sessions and replaces the old App Settings "Save key mappings" button (which only dumped the current bindings to `KeyMap<mode>.xml` for hand-editing) with a proper in-app **Edit Key Mappings** dialog. Applies to both **ioSender** and **ioSender XL**.

Console persistence: the three console checkboxes (*Verbose*, *Filter out realtime responses*, *Show all realtime responses*), whether the floating console window is open, and that window's size/position (clamped to the visible desktop so it can never restore off-screen) are all remembered. A new `ConsoleShortcut` setting (default Esc) toggles the console; it is now a first-class action inside the mappings editor rather than a separate, ad-hoc tab. A shared `ShortcutKey` helper (parse / storage / display) is used by both main windows, and an `AppConfig.ConsoleShortcutChanged` event re-registers the key live without a restart.

Key Mappings editor (`KeyMapEditor`, modal so jog/realtime dispatch can't fire mid-capture):

- One grouped, collapsible outline of every bindable action (grouped by function, like the Load Folder outline; groups remember their expansion per session). Click a row, press a combo to assign; right-click for *Reset to default* / *Clear*.
- Bindings changed from the factory default are highlighted; group headers flag a modified group and offer *Expand all* / *Collapse all* / *Reset group*.
- Per-axis jog keys are folded into the same list; rows and group headers carry tooltips.
- A *No-numpad keys* preset remaps the NumPad jog/step/feed actions to `Ctrl+1-8` and `[ ] - =` for keyboards without a numeric keypad (Windows exposes no reliable API to detect numpad presence, so this is a user-triggered preset).

The `KeyPressHandler` gains an editor API (`GetActionBindings/GetJogBindings/Apply...`) over its existing function catalog, identifying bindings by reflection method name with a friendly-label map. Persistence still uses the existing `SaveMappings/LoadMappings`.

#4 Fix unhandled exceptions on connection loss (e.g. SD card swap)

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	5	0
CNC Controls/CNC Controls/SDCardView.xaml.cs	13	0
CNC Core/CNC Core/Comms.cs	74	0
CNC Core/CNC Core/EltimaStream.cs	6	0
CNC Core/CNC Core/Grbl.cs	13	4
CNC Core/CNC Core/GrblViewModel.cs	45	0
CNC Core/CNC Core/SerialStream.cs	179	42
CNC Core/CNC Core/TelnetStream.cs	105	7
CNC Core/CNC Core/WebsocketStream.cs	79	8

When the controller's serial/USB device disappears — for example a board that resets and re-enumerates when its SD card is removed or reinserted — the status-poll timer kept writing to the dead port. The write threw (an `IOException`, then an `InvalidOperationException`: "BaseStream only available when the port is open") on a `System.Timers.Timer` thread, and the unhandled exception terminated the whole process. Opening the SD card tab afterwards threw similarly from `PurgeQueue`.

Changes:

- Guard all stream writes (`IsOpen` + try/catch). A device-gone fault now closes the port and starts an auto-reconnect instead of crashing.
- Add a shared, transport-agnostic `Reconnector` (used by serial, network and websocket) exposing `ConnectionLost` / `Reconnected` events. `GrblViewModel` handles them (stop/resume polling, show status); it is wired up at connect time in `AppConfig`.
- **Serial:** refactor open into a reusable `OpenPort()` that probes `GetPortNames()` and re-opens; publish the port only once it is actually open, to avoid a non-null-but-closed race; make `PurgeQueue` tolerant of a closed port; harden the receive handler against a port torn down mid-read.
- **Telnet / websocket:** guarded writes, loss detection (read returning 0 / `OnClose`) and reconnect via the same helper.
- Wrap the poll-timer callback in try/catch as a last line of defence.
- Guard the SD card view against acting on a closed connection.

#5 Add Stepper calibration (scratch) tab to grbl:Settings

File	+	-
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/GrblConfigView.xaml	3	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	1	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	92	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	486	0
.gitattributes	3	2
CNC Controls/CNC Controls/LibStrings.xaml	1	0
CNC Core/CNC Core/CNC Core.csproj	6	0
ioSender XL/ioSender XL/App.config	6	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	168	0

Adds a new tab to grbl:Settings for V-bit *scratch-line* steps/mm calibration over a long span — a more accurate alternative to the short jog-based calibration for the X and Y axes.

It generates an .nc program that scratches paired lines (perpendicular to the calibrated axis) for several candidate steps/mm values. Because GRBL cannot change \$100/\$101 mid-program, each candidate is simulated using the *commanded-distance trick*: the commanded distance $C = \text{span} \times \text{candidate} / \text{current}$, so the machine physically travels the distance that the candidate steps/mm value *would* produce.

The program is loaded into the job view (with an optional save to disk). After cutting, the user measures the actual spacings; the tab then computes and saves the corrected \$100/\$101 from those measurements (an implied value per pair, averaged). The prolog mirrors the Load Folder convention. X/Y only — Z continues to use the existing jog wizard.

Includes a small follow-up commit with high-DPI layout tweaks for the new tab.

#6 Add "Load Folder": load a folder of per-toolpath .nc files as one outlined program

File	+	-
.gitattributes	3	2
CNC Controls/CNC Controls/CNC Controls.csproj	2	0
CNC Controls/CNC Controls/FolderPicker.cs	159	0
CNC Controls/CNC Controls/FusionFolderLoader.cs	353	0
CNC Controls/CNC Controls/GCode.cs	156	0
CNC Controls/CNC Controls/GCodeListControl.xaml	45	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	129	0
CNC Controls/CNC Controls/JobControl.xaml.cs	4	0
CNC Core/CNC Core/GCodeJob.cs	41	5
CNC Core/CNC Core/GrblViewModel.cs	7	1
Fusion Addin/README.md	116	0
Fusion Addin/install-macos.sh	39	0
Fusion Addin/install-windows.ps1	40	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.manifest	11	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.py	476	0
ioSender XL/ioSender XL/MainWindow.xaml	1	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	5	0
ioSender/ioSender/MainWindow.xaml	1	0
ioSender/ioSender/MainWindow.xaml.cs	5	0
CNC Controls/CNC Controls/LibStrings.xaml	1	0
CNC Core/CNC Core/CNC Core.csproj	6	0
ioSender XL/ioSender XL/App.config	6	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	21	0
Locale/*/csv/ioSender.resources.*.csv (7 locales)	7	0

File > Load Folder reads a folder of per-operation files named <seq>_<name>_T<tool>.nc (as produced by the companion Fusion 360 add-in) and assembles them in memory into a single program:

- strips each file's header and program-end footer;
- inserts a G53 G0 Z0 safe-Z retract + M6 T<n> tool change before each toolpath, but only when the file does not already contain an M6 (so any post works);
- optionally restores rapid moves that Fusion Personal Use downgraded to G1;
- shows the result as a collapsible per-toolpath outline (groups default collapsed, and stay expanded while scrolling);
- adds *Start from this toolpath* and *Run just this toolpath* (the latter bounded to that section), each preceded by the G90 G94 / G17 / G21 prolog.

The new `FusionFolderLoader` is a faithful C# port of the add-in's combine / strip / rapid-restore rules; `FolderPicker` uses the modern `IFileOpenDialog` (`FOS_PICKFOLDERS`) for folder selection.

GCodeBlock gains a `Section` tag for outline grouping and an optional `AddLineNumbers` gate; GrblViewModel gains `IsFolderView` and a one-shot `RunToBlock` end-bound consumed by `CycleStart`.

Companion Fusion 360 add-in (ioSenderBatchPost): a standalone add-in (no external framework) that posts every operation in every setup to its own `<seq>_<name>_T<tool>.nc` file in a chosen folder — the input for Load Folder. Combining, tool-change insertion and rapid restoration are intentionally left to ioSender, so the add-in only posts. A post-processor dropdown (default `grbl.cps`) selects which post Fusion runs per operation. Ships with `install-windows.ps1` / `install-macos.sh` and a README. Fusion auto-discovers it once copied, but it must be enabled once via *Scripts and Add-Ins*.

Tool table for the simulator: the add-in also writes a `0_ToolTable.nc` carrying `(STOCK X= Y= Z=)` and `(TOOL T= D= TYPE= [A=])` comment lines — the stock box size and each tool's diameter and shape (FLAT/BALL/VBIT, with the V-bit included angle) — read from the setup's stock parameters and tool definitions. Load Folder detects this file (name matches `tool table`), loads it first with comments preserved, and pushes the leading `(STOCK ...)/(TOOL ...)` lines to a connected grblHAL simulator, which uses them for its 3D material-removal carve (real controllers ignore the comments). A dialog checkbox toggles it; the output matches the SRWCommands PostProcess add-in byte-for-byte. See `TOOL_TABLE_FORMAT.md` in the simulator repo.

Note: the `Fusion Addin/` folder is an optional companion tool; the maintainer may prefer to host it separately. It can be dropped from the PR if so.

#7 Improve high-DPI / large-text-size layout across the UI

File	+	-
CNC Controls Camera/ ... /ConfigControl.xaml	3	3
CNC Controls Lathe/ ... /ConfigControl.xaml	6	6
CNC Controls Probing/ ... /ProbingView.xaml	6	6
CNC Controls Probing/ ... /ProbingView.xaml.cs	3	1
CNC Controls/ ... /BasicConfigControl.xaml	4	4
CNC Controls/ ... /CoordValueSetControl.xaml	6	6
CNC Controls/ ... /FeedControl.xaml	5	5
CNC Controls/ ... /GotoBaseControl.xaml	5	5
CNC Controls/ ... /GotoControl.xaml	1	1
CNC Controls/ ... /JogBaseControl.xaml	2	2
CNC Controls/ ... /LEDControl.xaml	4	5
CNC Controls/ ... /NumericField.xaml	6	3
CNC Controls/ ... /NumericField.xaml.cs	3	1
CNC Controls/ ... /OutlineBaseControl.xaml	8	4
CNC Controls/ ... /OutlineControl.xaml	1	1
CNC Controls/ ... /OutlineFlyout.xaml	1	1
CNC Controls/ ... /OverrideControl.xaml	3	3
CNC Controls/ ... /SpindleControl.xaml	8	8
CNC Controls/ ... /StatusControl.xaml	3	3
CNC Controls/ ... /StepperCalibrationWizard.xaml	2	2
CNC Controls/ ... /StripGCodeConfigControl.xaml	15	9
CNC Controls/ ... /ToolView.xaml	2	2
CNC Controls/ ... /TrinamicView.xaml	1	1
CNC Controls/ ... /WorkParametersControl.xaml	18	18
ioSender XL/ioSender XL/JobView.xaml	6	6
ioSender/ioSender/JobView.xaml	1	1

Makes the UI usable at 125–150% Windows text scaling by replacing fixed control heights/widths with auto-sizing plus `MinWidth/MinHeight`, so labels and fields grow with the font instead of clipping or overlapping. Changes are layout-only — no behavioural changes.

- **Shared controls:** the `NumericField` label column now auto-sizes (`MinWidth = ColonAt`) with a `SharedSizeGroup` so labels align in any container that sets `Grid.IsSharedSizeScope`. `CoordValueSetControl`, `StatusControl`, `LEDControl` (dropped a hard `MaxHeight`) and `OverrideControl` no longer pin `Height/Width`; the **Spindle** group switches its fixed `Width=250` to `MinWidth` so the RPM field is no longer clipped at larger text sizes.
- **grbl:Settings config controls** (Basic / StripGCode / Lathe / Camera) and both Stepper calibration tabs auto-size their rows and the Axis dropdown; the Probing view auto-sizes its left panel with shared-size label alignment.
- **Main window** (JobView XL + ioSender): right-panel grid / columns / panels auto-size instead of fixed 515 / 250 px, and each panel control (WorkParameters, Spindle, Coolant, Outline, Feed, Goto)

auto-sizes.

#8 Native Xbox controller support (+ Key Mappings editor polish)

File	+	-
CNC Core/CNC Core/XInput.cs	160	0
CNC Core/CNC Core/ControllerService.cs	186	0
CNC Core/CNC Core/ControllerMapper.cs	408	0
CNC Core/CNC Core/CNC Core.csproj	9	0
CNC Core/CNC Core/GrblViewModel.cs	16	0
CNC Core/CNC Core/KeyPressHandler.cs	23	0
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	5	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	222	79
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	958	606
.gitattributes	3	2
ioSender XL/ioSender XL/App.config	6	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	259	0

Adds first-class game-controller support with **zero new dependencies** — the project uses no NuGet, so the controller is read via native **XInput** P/Invoke (xinput1_4.dll, falling back to xinput9_1_0.dll). A ~60 Hz DispatcherTimer poll layer runs in parallel to the keyboard handler.

- **Interop / service:** XInput.cs wraps the gamepad state and vibration structs; ControllerService.cs scans all four slots, edge-detects button down/up, and exposes Connected/Disconnected events, deadzone/normalize helpers and rumble.
- **Button map:** ControllerMapper.cs dispatches a default, fully re-mappable button map to machine actions (A = Cycle Start, B = Feed Hold, X = Spindle Stop, Y = Home, Back = Reset, Start = Unlock, D-pad = step-jog X/Y). Actions route through the same GrblViewModel.ExecuteCommand path as the on-screen buttons and are gated to a live connection in Idle/Jog/Tool states (with a status message when ignored). A short rumble confirms connection.
- **Analog jog:** the left stick proportionally jogs X/Y and the triggers jog Z, via throttled overlapping \$J= moves (jog-cancel on release) so motion stays smooth; speed scales with stick deflection. Enable, left-stick deadzone, max-feed multiple of the Jog panel feed, and per-axis invert are user-configurable. The button map and analog settings persist to ControllerMap.xml.
- **Controller tab** in the Key Mappings editor: live press indicator, per-button action dropdowns (each button's default marked with a leading *), *Restore Defaults* (enabled only when modified), machine-direction-aware tooltips, and the analog-jog settings — machine dispatch is paused while the dialog is open. New JogDistanceProvider/JogFeedProvider hooks let controller jog match the on-screen Jog panel's current step and feed.

The same branch also carries the keyboard-tab polish (changed-binding highlight with per-row/group reset, *Clear* moved into the row menu, column sizing, and the grouped outline now keeps a section expanded while scrolling instead of collapsing it). The branch is fully LF-normalized, so the counts above are the real diff — no line-ending noise.

#9 Configurable main page + flyouts + jog panels (ioSender XL)

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	141	20
CNC Controls/CNC Controls/AppConfigView.xaml	3	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	168	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	2	0
CNC Controls/CNC Controls/CNC Controls.csproj	65	0
CNC Controls/CNC Controls/Converters.cs	17	0
CNC Controls/CNC Controls/JogBaseControl.xaml	2	2
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	157	2
CNC Controls/CNC Controls/JogConfigControl.xaml	7	7
CNC Controls/CNC Controls/JogSlider.xaml	66	0
CNC Controls/CNC Controls/JogSlider.xaml.cs	51	0
CNC Controls/CNC Controls/JogUiConfigControl.xaml	18	9
CNC Controls/CNC Controls/KbdJogGridControl.xaml	46	0
CNC Controls/CNC Controls/KbdJogGridControl.xaml.cs	64	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml	25	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml.cs	34	0
CNC Controls/CNC Controls/LimitsControl.xaml	6	24
CNC Controls/CNC Controls/LimitsControl.xaml.cs	77	0
CNC Controls/CNC Controls/MachinePositionControl.xaml	69	0
CNC Controls/CNC Controls/MachinePositionControl.xaml.cs	21	0
CNC Controls/CNC Controls/MainPageEditor.xaml	108	0
CNC Controls/CNC Controls/MainPageEditor.xaml.cs	288	0
CNC Controls/CNC Controls/MainPanelRegistry.cs	111	0
CNC Controls/CNC Controls/NumericField.xaml	12	2
CNC Controls/CNC Controls/NumericField.xaml.cs	38	0
CNC Controls/CNC Controls/OffsetFlyout.xaml	41	0
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	133	0
CNC Controls/CNC Controls/PanelFlyout.xaml	43	0
CNC Controls/CNC Controls/PanelFlyout.xaml.cs	80	0
CNC Controls/CNC Controls/Properties/Settings.Designer.cs	36	0
CNC Controls/CNC Controls/Properties/Settings.settings	9	0
CNC Controls/CNC Controls/SidebarItem.cs	23	4
CNC Controls/CNC Controls/TabRegistry.cs	42	0
CNC Controls/CNC Controls/ToggleControl.xaml	20	2
CNC Controls/CNC Controls/UIJogGridControl.xaml	65	0
CNC Controls/CNC Controls/UIJogGridControl.xaml.cs	80	0
CNC Controls/CNC Controls/UIJoggingControl.xaml	44	0
CNC Controls/CNC Controls/UIJoggingControl.xaml.cs	29	0

CNC Core/CNC Core/KeyPressHandler.cs	22	4
CNC Core/CNC Core/SerialStream.cs	1	1
ioSender XL/ioSender XL/JobView.xaml	23	14
ioSender XL/ioSender XL/JobView.xaml.cs	100	19
ioSender XL/ioSender XL/MainWindow.xaml	3	3
ioSender XL/ioSender XL/MainWindow.xaml.cs	123	5
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	322	0

ioSender XL only. Makes the crowded right-hand panel area configurable instead of fixed. A new **Edit Main Page** dialog (beside *Edit Key Bindings*) is a shuttle editor with three buckets — *Available*, *Main page* (up to six slots), and *Flyouts* — so each named panel can be placed on the main page or as a pinnable right-edge flyout, or left off entirely to reclaim space. Nothing is auto-assigned.

- **Flyouts** share a flush, tab-height border and distribute their content vertically; each has a pin (stays open when another flyout opens, and persists across launches) and a close button.
- **Per-offset flyouts** (G28 / G30 / G54..G59.3 / G92) are tiny *Go* buttons (tooltip shows the live coordinates; G28/G30 also get a *Set* = G28.1/G30.1) so several can be pinned beside the jog panel.
- **Jog panels** — UI jogging (distance + speed) and keyboard default-speed — become assignable panels rendered as compact sliders with a read-only value; Settings:App stays the editor. The jog-arrow radios are hidden only when the UI jog panel is assigned.
- **Settings:App** gains opt-in autosave (with a discard prompt) and a *Reset to default* context-menu entry on numeric fields; the old *Edit Key Mappings* button is renamed **Edit Key Bindings**.

Layout choices persist (panels / flyouts / pins serialized as CSV to sidestep the `XmlSerializer` list-append quirk). Also includes robustness fixes: a resilient `SerialStream.OutCount` that no longer throws on a dropped port, and a deferred main-panel build so startup can't race the config load.

Extended: a fourth bucket places panels *left* of the 3D view (DRO, Program limits, Machine Position now in the registry) in a scroll region that never overlaps the fixed Signals/Status block. Per-offset *Set* now also handles G54–G59.3 (captures the current machine position as that coordinate system's origin via G10 L2 P<n>). The UI jog panel renders as a 2×4 distance/speed grid with green-click selection, a *Continuous* toggle and a *Remember distance & speed across restarts* option; *Link distance and speed to UI jogging* makes keyboard jogging follow the UI distance *and* speed (hiding the Keyboard Jogging panel). Coolant toggles render as a self-contained red/green pill (no theme dependency). A new **Tabs** tab in the editor reorders / hides the main TabControl tabs (Settings:App protected), applied on startup via `TabRegistry / Config.Tabs`. The *Restart* button moved to Settings:App (next to *Edit Main Page*), enabled only after a layout/tab change.

Panel refinements: the Machine Position panel shows MPos as a live `work + WorkPositionOffset` sum (tracking the DRO notifications); the limits panel stays visible and shows the machine soft-limit envelope from \$13x/\$23 when no program is loaded (program bounding box when one is), retitling itself *Machine limits / Program limits* accordingly; the 2×4 jog grid keeps its green selection highlight using the retained discrete index (survives keyboard/continuous-mode flips); and the **Tabs** editor now also lists tabs that host an `IGrbLConfigTab` (no `ViewType`) by falling back to the `TabItem` name.

#10 Macro manager + run-only flyout

File	+	-
CNC Controls/CNC Controls/MacroManagerDialog.xaml.cs	316	0
CNC Controls/CNC Controls/MacroManagerDialog.xaml	81	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	19	8
CNC Controls/CNC Controls/AppConfig.cs	19	0
CNC Controls/CNC Controls/AppConfigView.xaml.cs	10	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml	8	16
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Core/CNC Core/GCode.cs	5	0
CNC Controls/CNC Controls/JobControl.xaml.cs	3	2
CNC Controls/CNC Controls/MacroToolBarControl.xaml.cs	3	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	126	0

Adds an **Edit Macros** button to the Settings:App page opening a macro manager — a DataGrid with one row per macro, in the spirit of Fusion's *Scripts and Add-Ins* dialog — and reworks the on-screen macro flyout for running.

- **Name** and a **Prompt** (confirm-before-run) flag are edited in-line; an assignable **Key** column (F1–F12) sets the function key (keys are unique; *Create* auto-assigns the first free one); a **File** column shows the referenced file for @<path> macros (full path on hover), blank for inline.
- **Edit** edits the macro's definition (inline G-code, or the @<path> reference line) and reads it back; **View** opens what it points to — the referenced file for an @<path> macro (created if missing, to author a new one), otherwise the code. **Create/Delete** add and remove rows.
- The **macro flyout** is now run-only: a macro-name dropdown plus a **Run** button (the Edit button is gone — editing lives in the manager). It pre-selects the most recently run macro, persisted in `Config.LastMacroId` and updated from every run path (flyout, toolbar, F-keys), falling back to the first.

The macro's function key is decoupled from its Id via a new `Macro.FKey` field (legacy configs migrated on load: Id n keeps Fn). The @<path> *run-time* resolution — loading and running the referenced file, with directive processing — is provided by **#11**; this PR adds the manager/flyout support for authoring, viewing, and repointing references.

#11 Macro Processor Enhancements

File	+	-
CNC Controls/CNC Controls/MacroProcessor.cs	582	0
CNC Controls/CNC Controls/MacroToolBarControl.xaml.cs	1	1
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	1	1
CNC Controls/CNC Controls/JobControl.xaml.cs	1	2
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Core/CNC Core/GCodeParser.cs	31	12
CNC Core/CNC Core/NGCExpr.cs	22	0
CNC Core/CNC Core/Grbl.cs	11	0
CNC Core/CNC Core/GrblViewModel.cs	9	2

A sender-side **macro pre-processor** (`MacroProcessor.Run`) that interprets parenthesised directives before the remaining lines are streamed to the controller. All macro entry points — the top toolbar, the on-screen macro flyout, and the F-key handler — now route through it (previously the toolbar called `ExecuteMacro` directly and silently ignored every directive).

Directives, each a no-op the controller would treat as a comment, interpreted by the sender instead:

- **(PREREQ, cond, ...)** — require machine state *before* anything streams; if unmet the macro aborts with a message and nothing is sent. Conditions: `homed`, `idle`, `tlo`, `noalarm`, `connected`; stored positions / work offsets `g28`, `g30`, `g92`, `g54`–`g59.3` ("set" if any axis offset is non-zero, read fresh from `$#`); and any other token is required to be a controller `$I` build option (NEWOPT, e.g. `EXPR`), matched exactly and case-sensitively. `homed` reads `grblHAL's live sys.homed.mask` from `$#` rather than the change-based status `H: cache`, which can stay stale after a position-loss alarm.
- **(MBOX, [buttons,] message)** — modal hold; `OK/OKCANCEL/YESNO`, `Cancel/No` aborts the rest.
- **(WAITIDLE)** — hold streaming until the controller finishes what was sent and returns to `Idle` — needed after a controller-side job such as `$F=<file>`, which acks immediately and drops sender input while it runs.
- **(PROMPT param, default [, label])** — collect input values in a single up-front dialog and bind them two ways: assign the globals on the controller (so `$F=` files can read them) and substitute the references in the streamed body.
- **@<path>** — a macro whose body is a single `@<path>` line is a reference to an external file: that file's current contents are loaded and run in its place, re-read on every run — so a macro can be developed by editing the file directly. The loaded contents go through the same directive pipeline. (#10 adds the manager/flyout support for authoring and repointing these references.)

Supporting core changes so macros stream cleanly to an NGC-capable controller, all in the spirit of "don't reject what the controller can evaluate": `ExecuteMacro` now inherits the controller's expression support (as file streaming already did) and forwards verbatim any line its own parser cannot handle; `GCodeParser` recognises **named O-word subroutine calls** (`0<name> CALL [...]`) and forwards them to the controller, which resolves the named sub from its filesystem; and the `$# [HOME:...:mask]` `homed`-axes mask is parsed for the authoritative `homed` check.

#12 Onboarding tooltips

File	+	-
CNC Controls/CNC Controls/Converters.cs	50	0
CNC Controls/CNC Controls/StatusControl.xaml	3	0
CNC Controls/CNC Controls/StripGCodeConfigControl.xaml	10	5
CNC Controls/CNC Controls/JogUiConfigControl.xaml	10	9
CNC Controls/CNC Controls/JogConfigControl.xaml	7	7
CNC Controls/CNC Controls/BasicConfigControl.xaml	3	3
CNC Controls/CNC Controls/FileActionControl.xaml	4	4
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	196	0

Tooltips aimed at users new to CNC, where ioSender's UI is otherwise terse.

- **State field** — a context-aware tooltip explains the current controller state in plain language (Idle, Run, Hold, Check, Tool, Door, ...). For an **alarm** it appends the controller's specific message — e.g. ALARM 4: Probe fail ..., pulled from the GrblAlarms enumeration loaded at connect — plus how to recover, so a bare ALARM:4 is self-explanatory instead of a code to look up. (A new GrblStateToTooltipConverter drives a ToolTip binding on the State box.)
- **Settings:App** — a “what it does and why it’s useful” tooltip on each application setting (the grbl settings page already self-documents in its right-hand pane, so it’s left alone). Keyboard jogging (fast/slow/step feed & distance, link-step-to-UI), UI jogging (the four distance and four speed presets, noting distance and speed are selected *independently*), and GCode command stripping (M6/M7/M8/G61-G64, for machines lacking the matching hardware). Also fixes the “Keep MDI focus” tooltip (it had been copy-pasted from the buffering option) and the circular “Send comments” one.

#13 Build documentation: Mega-XL upgrade notes + repeatable tool-change guide

File	+	-
docs/Mega-XL-Upgrades.html	840	0
docs/Mega-XL-Upgrades.pdf	bin	0
docs/Repeatable-Tool-Change.html	419	0
docs/Repeatable-Tool-Change.pdf	bin	0

Two self-contained HTML documents (each rendered to a companion PDF), added under a new docs/ folder. Purely additive; touches no source.

- **Repeatable-Tool-Change** — a turnkey, novice-friendly tool-change guide built around a **G59.3 fixed toolsetter** and a T<n>/M6 flow, with a top-down machine travel diagram (color-coded paths). Aimed at the goal of repeatable, hands-off tool changes for users new to CNC.
- **Mega-XL-Upgrades** — build notes for an upgraded Kickstarter v1.0 Mega V XL: frame/spindle/motion/probing/power sections, a back-of-enclosure two-panel harness model (hand-built SVG diagrams), a cable & wiring schedule, and an **interactive bill-of-materials worksheet** — an in-page editor (double-click to edit Qty / Unit / Description / Source, live extended & category subtotals & grand total, URL shortening, Save-to-download) plus a linked table of contents.

The Mega-XL notes are specific to one machine and may be of limited upstream interest — they can be hosted separately or dropped from the PR. The repeatable tool-change guide is more generally applicable. The associated controller macros (`tc.macro`, `probe_tfl.macro`) and the ATC provisioning UI are intentionally *not* in this PR — they remain with the macro work, pending firmware support.

#14 SD + LittleFS file browser & file manager on the SD Card tab

File	+	-
CNC Controls/CNC Controls/SDCardView.xaml.cs	505	111
CNC Controls/CNC Controls/GrblFilesystems.cs	142	0
tests/GrblFilesystemsTests.cs	83	0
CNC Core/CNC Core/TelnetStream.cs	54	14
tests/README.md	22	0
CNC Controls/CNC Controls/SDCardView.xaml	19	6
CNC Core/CNC Core/YModem.cs	14	4
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	105	0

The SD Card tab listed only one filesystem (the SD card, or whatever was the current filesystem). On grblHAL builds that have both an SD card and a LittleFS store, the other filesystem — and any macros on it — were invisible.

This enumerates every mounted filesystem via `$FI ([FS:<name>@<path> size, free])` and lists them together in one grid with a new **Location** column, plus a per-filesystem **free-space banner** above the list. The right-click menu is reworked from *Run / Download and run / Delete* into a small **file manager**, and file operations target the file's own filesystem — a bare name on the root volume (unchanged single-SD behaviour) or an absolute path on a sub-mount such as `/littlefs`.

- **File manager menu:** *View / Edit / Create* (new content, or from a local file) open the file in Notepad — Edit and Create write the result back to the controller when Notepad closes; *Load* and *Load and run* bring the program into ioSender; *Copy to / Move to* transfer a file to another mounted filesystem (submenus built live from the mounts); *Delete*. Actions are enabled only for a real selected file.
- **Empty filesystems are visible:** a mount with no files gets a placeholder row so it can still be selected as an upload / copy target; the listing's re-entrant `Load()` is guarded (it pumps the dispatcher while waiting on the controller, so an overlapping refresh could corrupt the shared table).
- **Correct attribution under nested mounts:** a root `$F` recurses into sub-mounts, so an SD-root listing also returned the `/littlefs/*` files; each result is now attributed to its deepest containing mount, so files are not duplicated under (and mis-pathed on) the parent filesystem.
- **Upload to any filesystem:** `UploadLocalFile` targets a chosen filesystem via `$CWD` (the FTP path is built from the explicit destination, not the possibly-stale `FsCwd`), and upload is allowed on a LittleFS-only controller (no SD card present, so no SD mount status to require).
- **Upload reliability:** `TelnetStream` now writes *synchronously* — overlapped `WriteAsync` calls silently dropped a YModem block's payload/CRC — and `PurgeQueue` clears the async-read buffer instead of racing a second synchronous read on the same stream (which lost per-block ACKs, producing 0-byte files and multi-minute uploads); plus a bounded connect timeout so a reconnect retries quickly. YModem per-block ACK timeouts are raised (10 s open / 5 s block) to ride out flash-filesystem GC stalls, and blocks are sent in **128-byte (SOH)** packets rather than 1024-byte (STX): grblHAL's YModem receive ring is 1024 bytes (usable 1023), so a full 1029-byte STX packet can't fit — over telnet it arrives in a TCP burst faster than the firmware drains it and overflows,

stalling the transfer (a one-block file uploaded, larger ones hung). 128-byte blocks fit with margin on any transport.

- **Backward compatible:** builds that don't implement `$FI` (or report nothing mounted, `error:65`) return no `[FS:...]` lines, and the view falls back to the original single-filesystem mount + `$F` listing untouched. An SD card with files in its root works exactly as before, now also showing free space.
- **New `GrblFilesystems.cs`** holds pure, dependency-free helpers (mount-line parser, human-readable size, path qualification, free-space summary) and the `FsMount` model; the live `$FI/$F` querying lives in `GrblSDCard`.
- **Unit tested:** `tests/GrblFilesystemsTests.cs` exercises the pure helpers (29 assertions, run via the Framework `csc` — see `tests/README.md`; the app has no test project).

Builds clean (CNC Controls and the full ioSender solution). Also the groundwork (and the reliable LittleFS upload path) that the ATC macro-provisioning work reuses to target the `$FI`-discovered LittleFS path instead of a hardcoded one.

#15 USB / Ethernet / simulator connection support + Connect menu

File	+	-
CNC Controls/CNC Controls/SimulatorManager.cs	474	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	208	14
CNC Controls/CNC Controls/AppConfig.cs	165	28
CNC Controls/CNC Controls/BasicConfigControl.xaml	1	0
CNC Controls/CNC Controls/PortDialog.xaml.cs	56	2
CNC Controls/CNC Controls/PortDialog.xaml	43	8
ioSender XL/ioSender XL/MainWindow.xaml	39	2
CNC Controls/CNC Controls/GrblConfigControl.xaml.cs	22	0
simulator/README.md	18	0
CNC Core/CNC Core/TelnetStream.cs	14	3
CNC Controls/CNC Controls/JobControl.xaml.cs	14	2
CNC Controls/CNC Controls/UIUtils.cs	12	4
ioSender XL/ioSender XL/ioSender XL.csproj	12	0
CNC Core/CNC Core/HelperClasses.cs	8	2
ioSender XL/ioSender XL/JobView.xaml.cs	18	2
CNC Core/CNC Core/GrblViewModel.cs	7	0
readme.md	2	0
CNC Controls/CNC Controls/CNC Controls.csproj	2	0
CNC Controls/CNC Controls/Widget.cs	1	1
.gitignore	5	0
CNC Controls/CNC Controls/GrblConfigControl.xaml	1	0
simulator/sim-build.json	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	28	0

Adds first-class connection setup to the sender: pick USB (serial), Ethernet (`host:port`, now accepting hostnames as well as IPv4), or a local grblHAL **simulator** from a tab in the connection dialog, and (re)connect at any time from a menu. (*Updated 2026-06-20: the simulator is now run standalone and connected to over a port; the launch/download/My-Machine dialog was stripped out, and connection-loss + comment-forwarding fixes were folded in.*)

- **Simulator tab — connect-only:** the simulator now runs *standalone* — the user launches `grblHAL_sim.exe` directly (it opens its own 3D view + config window and listens on a TCP port), and `ioSender` simply connects to `127.0.0.1:<port>`. The tab is reduced to a single *Port* field ("Launch `grblHAL_sim.exe` first, then connect"); `ioSender` no longer launches, downloads or manages the simulator process.
- **Connect / Reconnect:** a `File > Connect...` item opens the dialog; while connected it becomes *Reconnect...* (disconnects, then re-opens the dialog so a different target can be chosen). The app now stays open when started without a connection instead of exiting, so the menu is reachable.
- **Remembered network host:** the dialog's network tab defaults to the most recent IP that connected successfully (persisted as `Config.NetworkHost`) rather than always the mDNS name `grblHAL.local`. It starts empty; on a serial connection it is seeded from the controller's \$I-

reported IP if still unset — so after one USB session a later reconnect already offers the controller's real address. Empty falls back to `grblHAL.local`; the bundled simulator (127.0.0.1) is excluded.

- **Prefer network connection (opt-in):** a Settings:App checkbox (`Config.PreferNetwork`, default off). After a serial/USB connection whose \$I reports an IP, ioSender probes `<ip>:23` and, if it answers, automatically switches the live link to the network — status line: *"Connection migrated to network (ip:23)"* — falling back to the serial port if the network connect fails. The probe runs off the UI thread; the switch is deferred and guarded against re-entry, and only fires from a serial link with a known IP.
- **Target status & cues:** the status bar shows the current connection target (`GrblViewModel.ConnectionTarget`), **green** when connected and **red** when not. The connect dialog opens on the tab matching the active target (Serial / Network / Simulator, disambiguated via the `StartSimulator` flag) and **green**-bolds that header; the XL window background tints **pale yellow** while connected to the **simulator** (restored to grey on a real target or when disconnected), so a virtual machine is unmistakable.
- **"Not connected" on a dropped socket:** when the link drops (e.g. the simulator window is closed) the reconnect loop keeps the target name set, so the target box would otherwise still read connected. An `IsConnectionLost` trigger now turns the XL target box red and shows *Not connected* until the connection is re-established.
- **Stop the job on disconnect:** if the controller connection drops *during a running job*, the **client-side** stream is torn down (job state reset to idle) so the UI no longer hangs waiting on ok responses that will never arrive — previously this was handled cleanly only while idle. This is a Stop, not a feed-hold, and because the link is already gone it cannot command the controller; what the machine does physically on comms loss is up to the firmware.
- **Always send comments to the simulator:** g-code comments are forwarded to the simulator regardless of the app's *Send comments* setting — the simulator reads them for tool/stock metadata (`(TOOL ...)`, `(STOCK ...)`) that drives its 3D view and material-removal carve.
- **Startup ordering:** `AppConfig.SetupAndOpen` is split into `LoadConfig` (run synchronously at construction, so config is available before any control's `Loaded` handler reads it) and `OpenConnection` (deferred to `ApplicationIdle` so the main window paints before the — possibly blocking — connection dialog). Transport is chosen by target string (`COMx` vs `host:port`) rather than a leading-digit IPv4 check.
- **Copy current settings to the simulator:** a *Copy to simulator* button on the `grbl:Settings` tab (next to Save) mirrors the live controller settings into the bundled simulator's "My Machine" EEPROM via `SimulatorManager.BuildMyMachineEeprom`, so a later simulator connection boots with this machine's configuration.

Builds clean (CNC Controls and the full ioSender XL + classic solutions) off **master**. The "My Machine" EEPROM builder is now wired to the *Copy to simulator* button (above); the `SimulatorManager` launch / web-builder-download machinery is no longer wired to the (now connect-only) dialog — a candidate for removal in a follow-up. No simulator binary is committed; see `simulator/README.md`.

#16 Validate controller (check-mode g-code conformance test)

File	+	-
CNC Controls/CNC Controls/ValidateProcessor.cs	1091	0
CNC GCodeViewer/CNC GCodeViewer/Renderer.xaml.cs	81	16
ioSender XL/ioSender XL/MainWindow.xaml.cs	3	21
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Controls/CNC Controls/LibStrings.xaml	3	0
ioSender XL/ioSender XL/MainWindow.xaml	1	1
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	21	0
Locale/*/csv/ioSender.resources.*.csv (7 locales)	7	0

Replaces the placeholder *Validate controller* action (#15) with a real, in-app conformance test: it streams a g-code program tailored to the connected controller's reported capabilities and reports which commands it accepts — without moving the machine.

- **Capability-tailored test:** generated from \$I build options, axis count, \$32 mode, tool count, aux-I/O counts and the work coordinate systems present, so a feature the firmware was never built with is not tested (and cannot be reported as a false failure).
- **Check mode, no motion:** streamed in \$C check mode and parsed only — the machine never moves, so homing and a clear envelope are not required. One feature per line, so an `error:N` pinpoints exactly which command the controller rejected.
- **Robust against grbl's check-mode behaviour:** a check-mode error *latches* the parser (every later line then errors), so the run recovers on each error — reset, re-enter check mode (unlocking first if the reset re-alarmed a homed machine), re-apply the modal set-up — then resumes, always re-confirming check mode before sending another line so a test line can never reach the controller as real motion.
- **Non-destructive:** check mode *commits* G10/G92 writes, so the run snapshots and restores the work offsets / tool table and avoids the un-restorable writes (G28.1/G30.1, and G92 when an offset is active). Machine settings (\$\$) are never written.
- **Progress & results:** a small non-modal panel shows a live pass/fail tally and the current test as it streams; on completion a *View Summary* button opens the full report (grouped by category, error code/message per failure, the pre-run parameter snapshot, Copy-to-clipboard). The controller is left in `Idle` with check mode off.
- **3D visualization:** the passed bounded-motion features are assembled into a runnable program (absolute G90 moves at a fast feed) and loaded into the buffer, so the user can press Cycle Start and watch the toolpath run in the 3D view via the normal job path. The viewer also gains an *always-active machine scene* (work envelope, axes and a live tool cone shown when homed even with no program loaded), plus fixes to the tool/executed-trail animation subscription and a bounded executed-trail advance (an out-of-range block was an unbounded spin).

Right-click the **Target** status item → *Validate controller*. Builds clean; the only failures on a stock grblHAL are commands it genuinely does not implement (e.g. G90.1, G61.1, G64, M52; M56 needs parking-override enabled).

#17 Restore main window position across launches

File	+	-
ioSender XL/ioSender XL/MainWindow.xaml.cs	58	7
CNC Controls/CNC Controls/AppConfig.cs	2	0

The "Restore last window size on startup" option restored only the window *size*; the position was discarded (clamped to the top-left). This persists the position too.

- **Position persisted:** new `WindowLeft/WindowTop` settings, saved on close (using `RestoreBounds` when maximized so the un-maximized rectangle is stored) and restored on startup — but only when the saved rectangle lands on a currently-connected monitor (checked against the whole virtual desktop), so a window saved on a since-removed screen can't open off-screen.
- **Takes effect immediately:** the live "save" flag was only set at startup, so toggling the option mid-session did nothing until a restart. It now updates live and captures + persists the current placement the instant the option is enabled.

Builds clean. The size/position restore builds on the connection support's `CompleteStartup`; the underlying `KeepWindowSize` option is from the ioSender baseline.

#18 grbl:Settings — live edits, change highlight, revert & snapshots

File	+	-
CNC Core/CNC Core/Grbl.cs	356	15
CNC Controls/CNC Controls/GrblConfigControl.xaml	49	6
CNC Controls/CNC Controls/GrblConfigControl.xaml.cs	39	8
CNC Controls/CNC Controls/Widget.cs	25	14
CNC Controls/CNC Controls/RestorePointDialog.xaml	41	0
CNC Controls/CNC Controls/RestorePointDialog.xaml.cs	65	0
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	77	0

Orca/PrusaSlicer-style indication of which controller \$ settings you have changed, on the grbl:Settings **Basic** tab, plus live editing feedback, granular revert and automatic per-Save restore points. Applies to both the grblHAL settings tree and the classic-grbl grid.

- **Live edits in the left-hand list:** edits stage in a pending-changes map (`GrblSettings.PendingEdits`) that the list reads via a new `EditValue`, so the value and its highlight update *immediately* without mutating the controller value. Checkboxes/radios commit on change; text/numeric editors commit on lost focus. Pending edits are flushed to `Value` and cleared on Save; discarded on Reload/Restore.
- **Two highlight states:** **orange/bold** = unsaved edit (`IsEdited`); **blue** = saved to the controller this session but still different from the session-start value (`IsSavedModified`). Groups roll up the same way.
- **Revert from a right-click menu:** *Revert to value at startup* on a setting, and *Revert all in group to value at startup* on a group node (staged as a pending edit, written on next Save).
- **Snapshot on Save + Restore picker:** every Save writes a timestamped restore point (full settings dump in the Backup format, led by a ; changes (N): old→new comment block), per controller identity, keeping the last 20. The **Restore** button opens a picker listing restore points newest-first with a changed-count column and a per-row tooltip showing exactly what each Save changed; *Browse...* falls back to any backup file.
- **Baselines, cleanly separated:** `GrblSettingDetails` tracks the current value, the frozen *startup* value (highlight) and the *loaded* controller value (drives `IsDirty`). Making `IsDirty` mean "differs from the controller" rather than "changed" fixes a spurious "settings changed, save now?" prompt when reverting an edit back to the controller value.
- **Bitfield flag tooltip:** `BITFIELD/XBITFIELD` settings still display numerically, but hovering the value shows the set flags decoded to their labels (one per line, N/A entries skipped) via `GrblSettingDetails.BitfieldLabels`.

Builds clean off **master** (CNC Controls and the full ioSender solution); verified against the bundled grblHAL simulator. Per-setting "revert to factory default" is intentionally deferred — grblHAL exposes no per-setting default (only the global `$RST=$` reset-all), so factory revert would need a future firmware-side snapshot of as-flashed values.

#19 Console — inline terminal-style command prompt

File	+	-
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	117	0
CNC Controls/CNC Controls/ConsoleControl.xaml	28	4
CNC Core/CNC Core/GrblViewModel.cs	20	0
CNC Core/CNC Core/KeyPressHandler.cs	7	0
CNC Controls/CNC Controls/MDIControl.xaml.cs	5	16
CNC Controls/CNC Controls/MDIControl.xaml	1	1
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	42	0

Makes the **Console** tab read like a traditional terminal by adding a command input line at the bottom, while keeping the global single-line MDI strip for the other tabs (3D View, Program). A natural answer to "why is MDI a separate strip instead of a console prompt?" — now it can be both.

- **Reuses the MDI plumbing:** Enter routes the line through the existing MDICommand, which already echoes the command into the console's ResponseLog — so the typed command and its response read inline like a shell. No JobView changes.
- **Shared command history:** a single GrblViewModel.MDIHistory (newest first) is fed by both the MDI strip and the console prompt and surfaced in both — the strip's dropdown binds to it and the prompt recalls it with Up/Down (Esc clears). A command entered in either place is recallable in the other. (The view-model CommandLog is not populated for these entries, so it is not used.)
- **Same guards, auto-focus:** the prompt is locked out while a job runs (same as the MDI strip) and grabs focus when the console becomes visible (tab switch or the pop-out console window) so it's ready to type into.
- **Pop-out for free:** the prompt is self-contained in ConsoleControl, so the detachable console window (which hosts the same control) gets it too.
- **Output context menu:** right-click the response log for *Clear all*, *Clear up to here* (drops the clicked line and everything above it), and *Save...* (writes the in-memory log to a text file).
- **No jog conflict:** the keyboard-jog handler jogs whenever a key is *mapped* to jog (Up/Down are by default) *before* its per-key text-box guard, so the prompt's arrows would otherwise move the machine. ProcessKeyPress now forces no-jog when the focused element is a TextBox tagged "MDI", and the prompt is tagged accordingly — uniformly exempting both the MDI strip's edit box and the console prompt.

Builds clean off **master** (CNC Controls and the full ioSender solution); verified against the bundled grblHAL simulator: commands echo + execute, auto-focus, history recall (Up/Down step through prior commands), and an A/B test confirming Up jogs the DRO when a non-input element is focused but does not when the console prompt is focused. Keys are handled in PreviewKeyDown so the arrows reach the prompt rather than being consumed by the TextBox.

#20 3D view — Ctrl+click to jog + per-axis orientation

File	+	-
CNC GCodeViewer/CNC GCodeViewer/Renderer.xaml.cs	164	52
CNC Core/CNC Core/Grbl.cs	26	1
CNC GCodeViewer/CNC GCodeViewer/RenderControl.xaml	11	7
CNC Core/CNC Core/GCodeEmulator.cs	3	0
CNC Controls/CNC Controls/AppConfig.cs	2	0

Ctrl+left-click in the 3D or top-down (XY) view jogs the machine to the clicked XY position — a quick coarse-positioning aid that complements the jog buttons (park over a corner to set work zero, check alignment, touch-screen rigs).

- **Safe by construction:** the move always retracts **Z to the top of travel first**, then rapids to the new XY, so the traverse can never plough through stock or fixtures. Only X/Y move; plain left-drag still rotates the camera.
- **How it maps:** the click ray is intersected with the work Z=0 plane (HelixToolkit `UnProject`) to recover an XY target, converted to machine coordinates ($MPos = WPos + WCO$), clamped to the soft-limit travel exactly as the jog limiter does, and emitted as two queued `$J=G53` jogs at the controller's max rate (`$110-$112`) so it moves at rapid speed.
- **Guarded:** requires homed + idle + max travel set + soft limits (`$20`) on, with a status-bar reason when a prerequisite is missing. Gated by a `GCodeViewerConfig.ClickToJog` setting (default on).
- **Orientation fix (prerequisite):** the rendered machine frame — work envelope, floor grid and crosshair — now honors the `$23` homing-direction mask + force-set-origin *per axis*, so the view matches any home corner (e.g. top-back-left, X+/Y-/Z-) instead of assuming +X/+Y. Standard +X/+Y/-Z machines render identically to before. `GrblInfo.ForceSetOrigin` now reads the live `$22` bit 3 rather than the `$I OPT 'Z'` flag (absent on some builds, e.g. the simulator), so the orientation is correct even when the option string omits it. The camera bounding box in the always-on machine scene is derived from the same per-axis envelope, fixing a mirrored home corner in the 3D view.
- **Toolbar tidy-up:** the 3D-view toolbar tooltips now show on disabled buttons and are reworded for consistency, the *save view* button is no longer needlessly gated, and the *show job envelope* button enables only with a file loaded. A null-tokens guard in `GCodeEmulator` avoids a crash when the machine scene renders with no program loaded.

Builds clean off `master` (CNC GCodeViewer chain). Verified on the author's machine: Ctrl+click retracts Z then rapids to the clicked XY at the configured max rate, and the envelope/grid orient to the real home corner.

#21 grbl:Settings — Machine Setup Wizard

File	+	-
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	795	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml	278	0
CNC Controls/CNC Controls/MachineCatalog.cs	194	0
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	30	0
CNC Controls/CNC Controls/CNC Controls.csproj	8	0
CNC Controls/CNC Controls/GrblConfigView.xaml	4	0
CNC Controls/CNC Controls/AppConfig.cs	2	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	2	1
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	427	0

A guided, first-run **Machine Setup Wizard** tab in grbl:Settings that configures the machine-description settings a fresh controller needs — so a new machine can be described without hunting through the \$ list. Auto-selected when the machine looks unconfigured (no remembered machine, or \$130–\$132 all zero).

- **Single page, nothing written until applied:** the whole machine description is laid out on one page — machine selector, a clickable home-position picture, a per-axis table, and the remaining homing & limits questions — so it reads at a glance instead of an eight-step click-through. *Apply* writes the changes via `GrblSettings.Save()`.
- **Apply only when there's something to write:** Apply is disabled while the wizard's values already match the controller; the pending-change set recomputes on every edit (machine pick, field, reload). Numeric settings are compared by value, so a formatting-only difference (1 vs 1.000) is not flagged as a change. Hovering Apply lists the exact pending writes (\$id name: old → new).
- **Out of the way:** sits as the *last* grbl:Settings tab (rarely needed after initial commissioning), so Basic settings is the default tab.
- **Machine catalog:** a built-in catalog (`MachineCatalog.cs`) of common hobby machines as manufacturer → product → model drop-downs (or a custom X×Y), each carrying a sensible home corner and force-set-origin default; the first entry is a generic XYZ machine with limit switches. Picking a model pre-fills the page, which the user then confirms or tweaks.
- **Axis table:** an editable per-axis grid — travel limit (\$130–\$132), max rate (\$110–\$112), steps/mm (\$100–\$102), invert direction (\$3) and normally-closed limit (\$5) — edited in place.
- **Home corner:** a clickable isometric box — click the corner where the machine homes — sets the X/Y homing directions (\$23) and auto-enables force-set-origin (\$22 bit 3) on grblHAL; Z is fixed to the top of the gantry. This is the same convention that orients the 3D view (#20). The axis table also carries a per-axis **Home min (\$23)** checkbox kept in sync with the picture, so the homing direction is visible/editable per axis (Z's box is locked to “top”).
- **Homing & limits:** hard limits (\$21) default on, with the remaining global questions as a short list and the explanatory help moved into per-input tooltips. A right-justified *Firmware Info* button shows the live \$I build output in a non-modal dialog.
- **Remembered & forgettable:** the chosen machine is saved (`AppConfig.LastMachine`) and restored next launch — keeping the live controller values over the catalog preset on restore — and a *Forget machine* button clears it so the wizard reappears next time.

- **Correct write order & honest errors:** grblHAL rejects soft limits (\$20=1) unless homing is enabled (`error:10`). Apply now enables homing by the user's intent (writing \$22 even on a fresh controller where it is currently off, instead of gating on the live state) and writes soft limits in a second pass — after \$22 — since `GrblSettings.Save()` writes in ascending id order. Soft limits are only requested when homing will be on. On failure it lists the exact rejected \$ settings and the controller's reason rather than a generic "failed to write settings".

Builds clean off `master` (CNC Controls and the full ioSender solution). UI verified by the author against a live controller and the bundled simulator; interactive \$3 direction-verify and homing-cycle-order editing were intentionally left out of this first cut. The first-run gate that forces the wizard to the foreground until a machine is applied — and its skip-when-connected-to-the-simulator behaviour — depend on connection-side plumbing and ride along in the integration branch rather than this stand-alone PR. The "copy this machine into the simulator's My Machine profile" action now lives on the `grbl:Settings` tab (#15).

#22 Keyboard jog — keep active when Job-view focus drifts

File	+	-
ioSender XL/ioSender XL/MainWindow.xaml.cs	19	0
ioSender XL/ioSender XL/JobView.xaml.cs	4	1

Keyboard jogging only fired through `JobView.OnPreviewKeyDown`, which needs keyboard focus inside the Job view's tree. Interacting with a flyout, side panel or menu moves focus out, so the arrow keys stopped jogging until the view was re-focused (e.g. by switching tabs). This keeps jog alive on the Job page wherever focus has drifted.

- **Window-level forwarding:** the existing `MainWindow` tunneling `PreviewKeyDown` (always fires) forwards jog keys to `JobView.ProcessKeyPreview` when the Job view is the current view but is not keyboard-focused. The focused case is unchanged — `JobView` still handles it, including its MDI/DRO/spindle/work-params jog gates.
- **Skips text input:** forwarding is suppressed when focus is in any `TextBoxBase`, so typing in the MDI strip or console prompt never jogs.
- **Clean stop:** a mirrored `PreviewKeyUp` forwards the key-up too, so a continuous jog started while the view was unfocused still receives its release and stops (no text-input guard here — an in-progress jog must always cancel).

Builds clean off `master` (the full `ioSender` solution). Two-file change reusing the console-shortcut `PreviewKeyDown` hook already on `master`; `ProcessKeyPreview` is promoted to `public` so the window can forward into it.

#23 Go To — optional safe-Z (lift, traverse, descend)

File	+	-
CNC Controls/CNC Controls/GotoBaseControl.xaml.cs	34	8
CNC Controls/CNC Controls/AppConfig.cs	5	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	14	0

A **Go To** a work-coordinate origin (G54–G59.3), G28 or G30 was a single coordinated rapid — a straight diagonal in machine space that can drag Z through clamps, fixtures or tall stock on the way. This adds an optional safe-Z decomposition so the traverse always happens clear of anything standing up in Z.

- **Three-phase move:** with *Safe Z on Go To* enabled, each Go To becomes G53 G0 Z0 (retract to machine top), then G53 G0 X...Y... (traverse at the safe height), then G53 G0 Z... (descend to the target). Machine Z0 is the homed top — already top-minus-pull-off with force-set-origin — so it never re-trips the Z limit, and it needs no configured clearance height (it clears anything that physically fits under the gantry).
- **All Go To targets:** applies uniformly to the selected work origin and to G28/G30, read from the same \$#-populated coordinate table the work-origin button already used (so G28/G30 get true X/Y-then-Z, not a firmware diagonal). Rotary axes travel with X/Y; only Z is separated.
- **Guarded, with a clean fallback:** gated on the *Safe Z on Go To* option (Settings:App → Main, default on), the machine being **homed** (machine coordinates valid) and **soft limits** (\$20) on so the three rapids are envelope-checked. If any is missing it falls back to the original single coordinated move — no behaviour change for unhomed/soft-limits-off setups.

Builds clean off **master** (both apps). The setting lives in the Base app config so it shows in the always-present Main settings panel regardless of jog mode; grbl's junction deviation (~0.01 mm) means the corners are effectively full stops, so no dwell is needed between phases.

#24 comms — fix telnet/websocket UI deadlock

File	+	-
CNC Core/CNC Core/TelnetStream.cs	17	6
CNC Core/CNC Core/WebsocketStream.cs	12	4

Over a telnet or websocket connection the UI could freeze whenever a modal message box (e.g. a macro M0/MBOX hold) was open while polling continued — a lock-ordering **deadlock** between the read callback and the UI thread.

- **The deadlock:** `ReadComplete` ran a synchronous `Dispatcher.Invoke(DataReceived)` *while holding* `lock(input)`. The UI thread's `PurgeQueue` also takes `lock(input)`, so once the dispatcher was busy (a modal pumping its own message loop) the read callback waited on the UI thread while the UI thread waited on the lock — a classic cyclic wait that hung the app.
- **The fix:** extract the complete replies from the buffer *under* the lock, then release it *before* dispatching them to `DataReceived`. The serial and Eltima streams already dispatched after releasing the lock (via `BeginInvoke`); this brings telnet/websocket in line.

Builds clean off upstream **master** (2.0.47); a pure comms-layer fix, no UI or feature change. Surfaced while testing the ATC Start Job sequence (#25), whose install-probe MBOX hold kept an active telnet poll — exactly the deadlock window.

#25 ATC tool-change macro provisioning (Install ATC + Start Job)

File	+	-
CNC Controls/CNC Controls/AtcMacros.cs	369	0
CNC Controls/CNC Controls/SDCardView.xaml.cs	80	3
CNC Controls/CNC Controls/SDCardView.xaml	4	1
CNC Controls/CNC Controls/CNC Controls.csproj	11	0
CNC Core/CNC Core/Grbl.cs	14	1
CNC Core/CNC Core/YModem.cs	14	5
ioSender XL/ioSender XL/JobView.xaml.cs	6	0
ioSender/ioSender/JobView.xaml.cs	6	0
macros/tc.macro	105	0
macros/probe_tfl.macro	140	0
macros/start_job.macro	52	0
macros/cal.macro	3	0

Turnkey **repeatable tool change** for ATC controllers: ioSender ships the controller-side tool-change macro set and installs it to the controller's persistent **LittleFS**, then seeds an ioSender-side "Start Job" macro that drives the home → set-WCS → reference-toolsetter → run sequence. The novice only sets G28/G30/G59.3 and supplies a 3D probe + toolsetter.

- **Embedded macros + provisioning:** AtcMacros.cs carries cal/probe_tfl/tc.macro as embedded resources and uploads them to /littlefs over **YModem** (absolute /littlefs/<name> path so they land on the right filesystem even with an SD card mounted at root; a atc.sum SHA-256 sidecar detects drift / re-installs).
- **Explicit "Install ATC macros" button** on the SD Card tab is the *only* provisioning trigger — user-initiated, re-verifying the filesystem each click. Auto-on-connect/auto-on-show were removed after proving fragile against controller-I/O timing (could wedge the link or hang on close).
- **\$I capability parsing** (Grbl.cs): ATC=1 → HasATC, ATC=0 → AtcMacrosRequired (an ATC is configured but its tc.macro is missing — prompt to install + reboot), YM → HasYModem. A 2-arg YModem.Upload(path, remoteName) overload targets an absolute remote path.
- **Start Job macro** (both apps): seeded on connect to an ATC controller (@start_job.macro, run line-by-line through the macro pre-processor). The controller-side macros it CALLs stay on LittleFS, resolved by 0<cal> CALL / M6. The Start Job go-to G30/G28 moves are written as an explicit safe-Z sequence (lift to machine top, traverse X/Y, descend) rather than a bare diagonal rapid; the Z probes descend to -140 (below the spoilboard, within \$132).
- **Embed-on-Build:** CNC Controls.csproj disables VS's fast up-to-date check so edits to the external ..\..\macros*.macro embedded resources are actually re-embedded on a normal Build (the fast check ignores external files and would otherwise ship a stale copy).

Builds clean in Release for both apps; the full Start Job sequence was verified end-to-end on hardware (2026-06-18). Kept self-contained on PR 14 by seeding the Start Job macro without an auto function-key (that nicely needs PR 10's Macro.FKey); assign one in the Macro manager if wanted. The 0<cal> CALL error:81 fix lives in PR 11 (label/keyword gap-close) plus a firmware-side macro-search-path fix.

#26 Offsets — editable G28/G30 (Set rapids & stores)

File	+	-
CNC Controls/CNC Controls/OffsetView.xaml.cs	21	2

The predefined offsets G28 and G30 were read-only in the Offsets view — you could only store the *current* machine position. This makes them **editable**: type a target and *Set* now honours it.

- **Set rapids, then stores:** because G28.1/G30.1 can only capture the live position, *Set* first rapids the machine to the edited target with G53 G0, then stores it with G28.1/G30.1. A confirmation prompt spells out the move first (it physically moves the machine); a pending unit change is honoured around the move.
- **Everything else unchanged:** non-predefined work offsets keep their existing behaviour.

Builds clean off `master` (CNC Controls and the full ioSender solution). One-file change to `OffsetView`.

#27 Dialogs — center owner-less modal dialogs

File	+	-
CNC Controls/CNC Controls/MPGPending.xaml	1	0
CNC Controls/CNC Controls/AppConfig.cs	1	1
CNC Controls Probing/CNC Controls Probing/ProbeVerify.xaml	1	0
CNC Controls Probing/CNC Controls Probing/ProbingViewModel.cs	1	1

Two modal dialogs — **MPGPending** (the “MPG mode active” prompt shown during the connect handshake) and **ProbeVerify** (probe-connection check) — set neither `Owner` nor `WindowStartupLocation`, so they opened as independent OS-placed top-level windows with their own taskbar button instead of centering over `ioSender`.

- **Owner + CenterOwner:** each now sets `Owner` to the main window at the call site and `WindowStartupLocation=CenterOwner + ShowInTaskbar=False` in XAML, so they center over the client area and stay attached to the app — matching every other dialog in the sender.

Builds clean off `master`. The other modal dialogs already set an owner; the owned-but-not-centered ones (g-code transform / importer dialogs) were left as-is for now.

File	+	-
CNC Controls/ToolsView.xaml	33	0
CNC Controls/ToolsView.xaml.cs	135	0
CNC Controls/GrblConfigView.xaml.cs	65	20
CNC Controls/IGrblConfigTab.cs	4	1

Reorganises the top-level UI into a clearer commissioning flow.

- **Settings → Grbl + App sub-tabs:** the controller (\$) settings and the sender's own preferences are split into two sub-tabs instead of one long page.
- **New Machine Setup top-level tab:** an overview plus six colour-coded step-tabs, with a startup *gate* that leads a new user to the first incomplete step (step 6 queries the controller filesystem via `AtcMacros.GetStatus` for macro status). Reworks the Machine Setup Wizard (#21) into the step-tab form.
- **New Tools container:** a single home (`ToolsView`) for the commissioning/tuning tools — Stepper calibration & scratch, Surface Spoilboard (#29), Squareness (#31), Trinamic and PID — via the `IGrblConfigTab` contract; top-level Trinamic/PID dropped.
- **Inline grids:** a probe-definition grid (step 5) and a macro-status grid (step 6) surface state without leaving the wizard.

In the integration build. The bulk of the wizard body lives in #21; this entry covers the hosting/restructure (Tools container + sub-tabs + gate). Localization CSVs for the new/renamed tabs are still pending.

File	+	–
CNC Controls/SurfaceSpoilboardWizard.xaml	84	0
CNC Controls/SurfaceSpoilboardWizard.xaml.cs	589	0

Flattens the spoilboard with a boustrophedon raster, generated to the homed machine envelope.

- **Machine-referenced:** the operator touches off *Z only*; XY is referenced to the homed envelope (less an edge margin), so the raster can never overrun a soft limit no matter where Z was zeroed.
- **High-point touch-off:** jog anywhere to the board's highest point and set Z0 there; the cut still references XY to the machine corner automatically.
- **Reuse Z0 + Finish pass + abbreviated perimeter Dry run:** re-cut at the same Z0 (with a cutter/dust-boot prompt), reserve a light final pass, or trace just the perimeter to check extents.
- **Outline = leveling check:** pauses at each corner on the Z0 plane for a dial-gauge reading.
- Runs through the flow-controlled job streamer (Feed Hold / Stop stay live).

In the integration build; hardware-tested.

#30 Load Stock — corner-probe stock measurement

File	+	–
ioSender XL/LoadStockView.xaml	102	0
ioSender XL/LoadStockView.xaml.cs	721	0
ioSender XL/MachineControlPanel.xaml(.cs)	63	0
ioSender XL/MachineControlWindow.xaml(.cs)	96	0
macros/pcorner.macro	143	0

Measures real stock by probing its corners, then sets the work origin and reports geometry.

- **Corner probe via pcorner.macro:** an inline grblHAL NGC program calls the controller-side corner-probe macro per corner; probe feeds come from the selected 3D probe definition (no hardcoded feeds).
- **Outputs:** XY footprint, per-corner Z (flatness), squareness (skew + diagonal Δ), and stock thickness (top – spoilboard reference). A results popup draws the probed quad with corner angles.
- **Copy size:** one click puts X Y Z on the clipboard to paste into the Fusion ioSenderBatchPost add-in (#6) — round-trips the measured stock into the simulator (STOCK) line.
- **Floating run-control panel** (MachineControlWindow) so the probe run can be driven without leaving the tab.

In the integration build; hardware-tested (427×427 mm stock).

#31 Squareness (Auto Square) — ganged-gantry offset tool

File	+	-
CNC Controls/AutoSquareWizard.xaml	97	0
CNC Controls/AutoSquareWizard.xaml.cs	589	0

Facilitates Phil Barrett's auto-square *offset* method for a ganged, auto-squared gantry.

- **Drill an L:** peck-drills a reference L (corner + far-X + far-Y holes, at the framing-square arm lengths, centred on the bed with X/Y nudge to dodge dog/insert holes).
- **Measure & apply:** register a framing square on the pins, enter the gap; the tool converts it ($\text{gap} \div \text{lever} \times \text{rail span}$) to a squaring-offset change, writes \$170-\$172 and re-homes.
- **Ganged-axis selector:** grblHAL reports the offset for all of \$170-\$172 but only the ganged axis accepts a write — the selector targets the right one.
- **Measure-only gauge:** on a build without the offset setting it still drills + measures to show how far out of square the gantry is (correct mechanically). Dry run + Reuse Z0.

In the integration build. Note: the squaring offset is fine-trim only — a grossly racked gantry must be corrected mechanically first.

#32 Program view as a reusable object + background loading

File	+	-
CNC Controls/ProgramView.xaml(.cs)	129	0
CNC Core/GCodeJob.cs	108	49
CNC Controls/GCode.cs	99	33
ioSender XL/MainWindow.xaml(.cs)	79	44
ioSender XL/LoadStockView.xaml(.cs)	68	16
CNC Controls/GCodeListControl.xaml(.cs)	29	3
CNC Controls/{AutoSquare, SurfaceSpoilboard, StepperCalibration}Wizard.xaml.cs	57	9
CNC Controls/JobControl.xaml.cs	13	7
CNC Controls/MacroProcessor.cs	0	14
macros/pcorner.macro	13	8
docs/ (2 design/session notes)	224	0
CNC Core/GrblViewModel.cs, CNC Core.csproj	11	1

"A program is a program", taken further: the single shared program overlay becomes a standalone, streamer-connectable ProgramView object, and the loader stops blocking the UI.

- **Reusable program view:** Load File, Load Folder and each wizard create their own ProgramView and Connect() it to a push/pop stack — the overlay hosts whichever is active, and the loaded job is no longer a special "null" case. One Cycle Start runs the active view.
- **Continuous block numbering:** an explicit N word is shown in the Block column and hidden from the G-code column (Data stays intact for streaming); unnumbered lines (O-word calls, comments) continue previous + 1.
- **Background loading:** Load File / Load Folder parse on a worker thread — the per-line parse and the end-of-load bounding-box emulation run off the UI thread, blocks flush to the bound list in batches (rows appear as files are read), and a program-view-scoped wait cursor replaces the global one. A 322k-line folder combine no longer freezes the app.
- **Also shipped here:** Load Stock "set tool-length reference" option (Start Job parity) and the absolute-seek pcorner.macro (workaround for the G53-then-G91-on-inverted-axis firmware bug, filed upstream).

In the integration build; hardware-tested (Load Stock, single-file and folder loads, background load).

#33 Explain G-code on hover (novice tooltips)

File	+	-
CNC Core/GCodeExplainer.cs	634	19
CNC Controls/GCodeListControl.xaml(.cs)	170	20
CNC Core/CNC Core.csproj	1	0

Makes the program view teach: hovering a line shows a plain-language explanation of what the G-code does — aimed at novices.

- **Token-based:** builds the explanation from the parser's own semantic tokens (keyed to a block by line number), so it inherits the parser's modal state — a bare X10 Y0 continuation still reads correctly as a rapid or a cut.
- **Line or selection:** hovering one line explains it ("Cutting move to X 125.4, Y 0; feed 800 mm/min"); hovering a line within a multi-row selection gives a line-by-line breakdown.
- **Full coverage:** a complete G/M-code dictionary plus a raw-line lexer fallback explain bare modal codes (e.g. G54) and views without parsed tokens. Computed lazily per row; skipped (thread-safe) while a background load is still parsing.
- **Reads real programs:** a bracket/parameter-aware lexer explains expression operands (G53 G0 X[#5181]), decodes the well-known #5xxx parameters (G28/G30 positions, probe result, WCS offsets), names O-word CALLs by the /<name>.macro file they run + their arguments, and reads G10 (WCS origin/rotation), G65 P5 (probe-input select) and #-assignments plainly.
- **Click to copy:** the hover balloon is an interactive popup — click it to copy the explanation to the clipboard (a plain tooltip can't be moused into).
- **Localizable:** English for now, but every phrase is produced in one place so it can move to LibStrings later.

In the integration build; hardware-verified.

#34 Probe definition diagrams

File	+	-
CNC Controls/ProbeDefinitionEditDialog.xaml	136	22
CNC Controls/ProbeDefinitionEditDialog.xaml.cs	6	0

The probe definition editor now shows a labelled schematic beside the fields, so a novice can see what each measurement means.

- **Per-type drawing:** switched with the Type dropdown — 3D touch probe, touch plate, tool setter and edge finder each draw their shape and dimension the key measurements the user enters (tip / ball / bit diameter, plate thickness, lip width, setter height, spin speed).
- **Touch-plate lip:** drawn as a down-protruding hook whose inner face registers against the stock side (aluminium).
- **Pure vector** (Canvas + Viewbox), toggled with the same show/hide as the fields — no image assets, theme-friendly.

In the integration build.

#35 Jog distance +/- controller actions

File	+	-
CNC Core/ControllerMapper.cs	11	1
CNC Core/GrblViewModel.cs	2	0
CNC Controls/JogBaseControl.xaml.cs	1	0
CNC Controls/KeyMapEditor.xaml.cs	3	1

Two new assignable controller actions that step the on-screen jog *distance* preset (the 2×4 grid), distinct from the existing *Jog speed* +/- which step the feed preset.

- **Jog distance +/-** appear in the Controller-tab action list; assign them to any button (no default binding).
- Adds `GrblViewModel.CycleJogDistance`, wired by the jog panel to the distance index (clamped to the four presets).

In the integration build.

#36 Probe input follows the selected probe type

File	+	-
CNC Core/Grbl.cs	4	0
ioSender XL/LoadStockView.xaml.cs	6	0
ioSender XL/HeightMapView.xaml.cs	6	0

The program-generating tools now select the controller probe input from the chosen probe's type — the same rule the Probing page already uses — so a stale selection can't send a 3D-probe descent to the wrong input and drive into the work.

- **Rule:** Load Stock and Height Map emit G65 P5 Q<n> for the selected probe — tool setter → Q1, else the main probe → Q0.
- **Why:** an interrupted tool-setter cycle (`tc.macro`) can leave G65 P5 Q1 selected; a later probe would then watch the tool-setter line and stab the spoilboard.
- Gated on new `GrblInfo.HasToolSetter` (only meaningful when a tool setter exists to select between).

In the integration build; hardware-verified (Load Stock).

#37 One in-place streaming path (stayPut removed)

File	+	-
ioSender XL/MainWindow.xaml.cs	49	66
CNC Controls/MacroProcessor.cs	12	43
ioSender XL/LoadStockView.xaml.cs	1	1

With each program view self-contained, the old `stayPut` flag and the “stream-in-place vs load-into-job” fork were a relic of the single shared job view. Collapsed to one path.

- **One streaming path:** every streamed macro/wizard run streams a transient with full flow control (Feed Hold/Stop stay live) and **never touches the loaded job** (`GCode.File`) or the Job tab. Before, wizards other than Load Stock replaced your loaded program when they ran.
- **Own view per run:** a tool that owns a program view (a wizard) marks its own; a plain streamed macro gets a dedicated run view (titled with the macro name) that auto-dismisses ~20 s after it finishes.
- **Multi-burst safe:** a run flushes several bursts (a park move, then each `0< ... > CALL`); the streamer source reverts per burst, guarded so a finishing burst can’t clobber the next, and the view stays connected across all of them.
- Deletes the `stayPut` param, the `RunStreamedJob` job-clobber path + its wiring, and the orphaned tab-restore helper — a net code removal.

In the integration build; hardware-verified via a clean Load Stock run (multi-burst / directives / O-words). Not yet exercised on hardware: the plain-macro run view (only a large *streaming* flyout macro reaches it).

File	+	-
CNC Controls/SimulatorManager.cs	309	0
ioSender XL/JobView.xaml.cs	46	0

Keeps the bundled grblHAL simulator in step with whatever controller you connect to, so a probe / rotation / multi-axis feature behaves on the sim exactly as on the real firmware.

- **Options → signature:** on connect, the controller's behaviour-affecting build options (axis count, probe, WCS rotation) are read from `GrblInfo` into a set of compile symbols and a short 12-hex signature.
- **Cache ladder:** already-active build → locally-cached `grblHAL_sim-<sig>.exe` → a `sim-<sig>` release on the fork (public download, no token) → otherwise dispatch the parameterized `build-matched-sim` CI and pick up the result. The set of `sim-<sig>` releases acts as a shared remote build cache.
- **Token only to dispatch:** a GitHub token (`GH_TOKEN`, `workflow` scope) is needed only to trigger a build — every download is public. Runs on a background thread off `InitSystem`, never blocks connect, is skipped when talking to our own simulator, and won't re-dispatch an in-flight signature.

In the integration build; the Simulator-fork CI (`build-matched-sim.yml`) validated green end-to-end (dispatch → per-signature release → public download). The real on-connect round-trip is pending a hardware spot-check.

#39 Re-establish modal state on a mid-program start

File	+	-
CNC Controls/StreamPump.cs	22	0

“Start from this toolpath” queues a G90 G94 / G17 / G21 prolog to re-establish modal state (the folder combiner strips each per-op file’s header, where G21 lived). The legacy streamer drained that queue, but the `StreamPump` streams `source.Data` directly and never touched `source.Commands` — so the prolog was silently dropped on a mid-program start.

- **Symptom:** a mid-program start inherited whatever units the controller was left in; in G20 the toolpath’s literal-mm coordinates read as inches → targets off the table.
- **Fix:** the pump now sends the queued `source.Commands` lines first, ahead of the first block, and swallows their acks so they don’t advance job-line accounting. Full runs from the top were unaffected (there the prolog is prepended as real blocks).

In the integration build. A latent correctness fix for any pump-based run-from-block.

#40 Load Stock “apply work rotation” defaults off

File	+	–
ioSender XL/LoadStockView.xaml.cs	6	1

Setting a WCS rotation via G10 L2 R armed a grblHAL rotation-transform bug (a rotated G54 move with both X and Y double-applied the second axis’s offset) that soft-limited the *next* program’s first cut rapid. Reproduced on hardware: a -0.08° measured skew crashed the following job; clearing the rotation let it run.

- Default the skew→WCS rotation **off** so Load Stock never silently arms it; the checkbox stays for opt-in.
- The real fix is in the firmware rotation transform (`gcode.c: bit(idx)→bit(idx_0)`), now fixed and flashed; once hardware-verified the default can go back on.

In the integration build. A guard against the firmware rotation bug until the firmware fix is hardware-verified.

#41 Live 3D toolpath + material-removal carve view

File	+	-
CNC GCodeViewer/CarveView.xaml.cs	850	0
CNC GCodeViewer/Renderer.xaml.cs	190	54
CNC GCodeViewer/CarveView.xaml	36	0
CNC GCodeViewer/ConfigControl.xaml.cs	13	1
CNC GCodeViewer/RenderControl.xaml	11	7
CNC GCodeViewer/CNC GCodeViewer.csproj	7	0

A live 3D **carve view** in the toolpath viewer — not just lines on screen, but the stock being *machined away* as the program runs, so you can see the finished shape before you cut it.

- **Machine scene, always on:** the work envelope, axes and a live tool cone are shown whenever the machine is homed, even with no program loaded — a persistent sense of where the tool is in the work area.
- **Program-sized stock:** the stock block is sized to the program (its real footprint read from a (STOCK X Y) comment), drawn solid with the correct top, in work coordinates aligned to the toolpath.
- **Material-removal carve:** a dextral / height-field carve removes material as the tool moves, using the *real cutter bottom profile* (flat / ball / V-bit) on a fine grid, so the carved surface matches the actual tool. Toolpath lines hide during playback to reveal the cut and restore on stop.
- **Play a preview:** press Play for a machine-free preview that animates the program and builds the carve; Reset view restores the default orientation, and a pale stock theme keeps the cut visible.
- **Real machine vs. simulator:** streaming to a *real* machine **disables** the carve — the view freezes so it never competes with motion-critical streaming. To watch the carve run in step with execution, connect to the **simulator** and Cycle Start there.
- **Performance:** in-place dirty-cell mesh updates (no per-frame realloc) and the scene build deferred off the layout pass.

In the integration build. Built up over several phases (envelope / stock / tool cone → toolpath + Play → dextral carve); the validate-controller 3D visualization ([#16](#)) uses the same scene.

#42 Settings redesign — flat tab strip + searchable Grbl settings

File	+	–
CNC Controls/GrblConfigView.xaml.cs	359	56
CNC Controls/GrblConfigControl.xaml.cs	156	26
CNC Controls/KeyMapEditor.xaml.cs	66	40
CNC Controls/GrblConfigView.xaml	41	8
CNC Controls/BasicConfigControl.xaml	27	14
CNC Controls/GrblConfigControl.xaml	27	3
CNC Controls/MainPageEditor.xaml.cs	19	3
CNC Controls/MacroManagerDialog.xaml.cs	14	9
CNC Core/Grbl.cs	10	0
CNC Controls/SettingsPanelRegistry.cs	8	0
CNC Controls/AppConfig.cs	5	0
CNC Controls/AppConfigView.xaml(.cs) (retired)	0	373
Camera / Probing / Lathe / Jog config panels (widen + wrap)	19	10
Locale/*/csv (7 locales)	184	0

The Settings area was two tabs — **Grbl** and an **App Settings** page that was a wall of GroupBoxes plus a row of buttons that each opened a modal dialog. It's now a single **flat tab strip**:

Grbl · App · Jogging · G Code · Keyboard & Controller · Macros · Main Page.

- **Free-text Grbl search:** a leading \$ means “setting id” — \$53 jumps to it, \$5? expands all of \$50–\$59; anything else (a word like Jog or a bare number like 10) matches every setting whose *name* or *value* contains it. Matching groups auto-expand and matches are highlighted; an “**n of m**” readout plus up/down arrows (and **F3/F4**, Enter) step through them.
- **Grbl-settings autosave:** an opt-in toggle (default **off** — these are \$ writes to hardware) with a companion “prompt before auto-saving”; when on, the manual Save button on the Grbl tab hides, mirroring the App-settings autosave.
- **Editors folded in as tabs:** the Key Bindings, Macros and Main Page editors were modal Windows launched by buttons — they're now inline tabs (converted to `UserControls`) that *save on leave* instead of OK/Cancel. The Key Bindings tab only pauses controller dispatch while it is actually shown.
- **App/Jogging/G Code:** the config panels are bucketed by type into the three tabs; App & Jogging columns are 25% wider, Camera + Probing share one column, and panels stretch to fill their column so long labels wrap instead of clipping.
- **Under the hood:** `AppConfigView` is retired — its Save/Restart footer, restart-hooking and auto-save-on-leave moved into the settings host. New strings localized across all seven locales.

In the integration build. Builds clean (Release, EXIT 0); on-hardware review of the high-DPI arrow buttons and panel wrapping pending.

#43 Per-user config folder + standalone config folded into App.config

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	258	1
CNC Controls/CNC Controls/ConfigPanel.cs	97	0
CNC Controls/CNC Controls/LoadStockConfig.cs	37	0
ioSender XL/ioSender XL/Default-App.config	64	0
CNC Core/CNC Core/ControllerMapper.cs	82	27
CNC Core/CNC Core/KeyPressHandler.cs	102	63
CNC Controls/CNC Controls/ProbeDefinition.cs	32	4
CNC Controls/CNC Controls/SettingsPanelRegistry.cs	43	0
Surface/AutoSquare/StepperScratch Wizard .xml(.cs) - ConfigPanel conversion (3 wizards)	121	187
CNC Controls/CNC Controls/ConsoleControl.xml(.cs)	162	6
CNC Controls/CNC Controls/GrblConfigControl + GrblConfigView .xml(.cs)	143	63
Basic/StripGCode/Jog/JogUi ConfigControl + KeyMapEditor + JobControl	126	16
CNC Core/CNC Core/Grbl.cs, GrblViewModel.cs	2	2

App.config now lives per-user in %AppData%\ioSender: on first run the existing app-folder config (plus key maps, probe defs, macros and settings backups) is moved there, and the app-folder copy ships read-only as the reset-to-default template. The standalone .xml files each feature used to write next to it are folded in as App.config sections.

- **Section fold-in:** ProbeDefinitions, the game-controller map, key mappings, and the Surface Spoilboard / Auto Square / Stepper calibration / Load Stock parameter blobs become sections; a one-time importer reads any legacy file then deletes it. A new ConfigPanel<T> base gives the wizards save/restore of their section from a single DTO.
- **Curated defaults:** a shipped Default-App.config is the factory template (auto-saves default off) and the source for "Reset to default".
- **Settings/console polish:** the console gains an output search bar + a font-size control; the Settings tabs share one Save/Restart/Reset footer (Grbl sub-row for reload/backup/restore/copy-to-sim); panels opt into "Reset to default"; *ResetAndUnlock* is a bindable keyboard action; a key binding reads as "modified" only when it differs from its value at startup.

In the integration build. Builds clean (Release, EXIT 0); first-run migration is best-effort and idempotent. On-hardware migration spot-check pending.

#44 Start Job workflow + Verify skew, flyout & offset fixes

File	+	-
ioSender XL/ioSender XL/LoadStockView.xaml(.cs)	229	127
ioSender XL/ioSender XL/MainWindow.xaml(.cs)	88	12
ioSender XL/ioSender XL/JobView.xaml.cs	3	3
CNC Controls/CNC Controls/AtcMacros.cs	6	19
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	7	3
CNC Controls/CNC Controls/OffsetView.xaml.cs	26	9
CNC Controls/CNC Controls/LayoutModel.cs	2	1
CNC Controls/CNC Controls/LibStrings.xaml + Locale/*/csv (7 locales)	8	8

Load Stock is reworked into **Start Job** — the first tab, the guided front end for setting a run up before the Job tab cuts it.

- **Workflow:** the operational tabs are disabled until a controller connects; startup lands on the Job tab so its handshake reads \$I. Start Job auto-selects the configured 3D probe (no picker), zeroes at the front-left corner, and writes `start_job_macro` so the seeded macro/F-key runs the same sequence.
- **Verify skew:** a new pass re-touches each corner in the rotated work frame — the back corners twice (ideal rectangle, then the probed corner) so the gap reads out the stock's out-of-square amount — then parks at G30. The measured-skew rotation sign is corrected (`grblHAL G10 L2 R` aligns with `-atan2(dy,dx)`).
- **Flyout & offset fixes:** pinned sidebar flyouts stay open across tab switches; a flyout *Set* writes the offset directly (it was silently dropped by the streamer-state gate); the Offsets tab *Set All* skips the rapid-to-target confirmation when the machine is already there.

In the integration build. Builds clean (Release, EXIT 0). Start Job macro run + Verify skew validated on hardware; a separate 3D-view stock-orientation issue is tracked.

#45 Run-bar jog preset spinners + startup splash on top

File	+	-
CNC Controls/CNC Controls/JogPresetSelector.xaml(.cs)	106	0
ioSender XL/ioSender XL/MainWindow.xaml	11	2
ioSender XL/ioSender XL/SplashWindow.xaml	1	1
CNC Controls/CNC Controls/MDIControl.xaml(.cs)	14	6

Small quality-of-life fixes to the always-visible run bar.

- **Jog preset spinners:** a new compact selector fills the space on the *State / Check* line — *Dist [▼ value ▲] Jog [▼ value ▲] Feed*. Each field is read-only; the down/up arrows step through the four jog distance/feed presets and **wrap around** at the ends. It binds the shared *JogData* singleton, so the selection stays in sync with the jog pad, keyboard and controller (the preset values are still edited on Settings:App / the Jog tab). Right-aligned to the Program button's edge above.
- **Startup splash stays on top:** the progress splash was `Topmost="False"` and got buried by the main window's black frame almost immediately; it is now `Topmost="True"`, so the whole init sequence (connecting → reading settings → validating setup) is actually visible during the ~10–15 s startup.
- **MDI Send button re-aligned:** the MDI line was rebuilt as a single-row grid (command box + *Send*) that share one row height, and the run bar's *Bare* mode no longer pins *Send* to its own fixed height — so *Send* stays vertically centred against the editable command box (it had drifted after the high-DPI pass dropped its fixed height).

In the integration build. Builds clean (Release, EXIT 0). Splash confirmed on hardware; spinner layout and Send-button alignment confirmed in the running app.

#46 High-DPI audit — text no longer clips at 125–150% scaling

File	+	–
CNC Controls/CNC Controls/*.xaml (38 files)	114	115
CNC Controls Probing/CNC Controls Probing/*.xaml (10 files)	35	35
ioSender XL/ioSender XL/*.xaml (4 files)	18	18
CNC Controls Camera/CNC Controls Camera/*.xaml (2 files)	8	8
CNC Controls Lathe/CNC Controls Lathe/*.xaml (4 files)	7	7
CNC Converters/JobParametersDialog.xaml	6	6
CNC GCodeViewer/CNC GCodeViewer/*.xaml (3 files)	4	4
CNC Controls Dragknife/DragKnifeDialog.xaml	4	4

A reviewed, case-by-case scrub of hard-coded pixel `Width=`/`Height=` across the whole UI, so controls size to their content and no longer clip text when Windows text scaling is set to 125–150%. Roughly 250 attributes across 63 XAML files were classified one by one (not a blind find/replace):

- **Drop fixed Height** on text-bearing controls (buttons, labels, text boxes, combos, radio buttons, group boxes, and the rows that hold them) so they auto-size to the scaled font instead of cropping glyphs.
- **Convert fixed Width to MinWidth** (and floor heights to `MinHeight`) where the value was only a minimum — text buttons, combos, fields and container panels can now grow.
- **Convert fixed-pixel grid rows to Height="Auto"** where the row holds text.
- **Deliberately kept** genuine fixed geometry that must *not* scale with text: icons and images, LED/signal indicators, probe/lathe schematic diagrams, 3D render surfaces, colour swatches, icon-only glyph buttons, data-grid column widths, colon-aligned label columns, load-bearing scroll viewports and fixed dialog chrome.

In the integration build. Builds clean (Release, EXIT 0). High-traffic controls confirmed on hardware (“no more clipped text”). Known follow-ups: the five absolute-`Margin` lathe wizards (ThreadingWizard, ProfileDialog, Turning/Parting/Facing) need a real re-layout rather than an attribute scrub, and are intentionally left for a later pass.

#47 Lathe mode enable/disable checkbox

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	41	0
CNC Controls/CNC Controls/BasicConfigControl.xaml.cs	14	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	1	0

A *Lathe mode* checkbox on the App > Main settings panel, so lathe mode can be turned on or off without hand-editing the config. Previously the only lathe-enable control lived on a settings panel that a lathe view had to load first — and that view/tab was itself gated on lathe already being enabled, a catch-22 that made it unreachable when off.

- **Simple on/off facade** — `LatheConfig.LatheEnabled` maps to the existing `XMode` (Radius when on, Disabled when off) and persists through the same Lathe config section.
- **Shows/hides the Lathe wizards tab** — a new `AppConfig.SetTabPresent` adds or removes the tab in both the persisted layout tree and the legacy flat tab list, inserted after the SD-card tab.
- **Restart to apply** — the tab is only built at startup from the controller's reported lathe mode, so toggling raises `RestartRequired` (see [#48](#)).

In the integration build. Builds clean (Release, EXIT 0). Restores an implementation that had been prototyped then reverted; hardware-confirmed working.

#48 Restart button pulse + restart-on-leave prompt

CNC Controls/CNC Controls/GrblConfigView.xaml	43	1
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	15	0

Makes a pending restart impossible to miss. Some settings (buffer/reset delay, 3D-view enable, main-page layout, and now lathe mode) only take effect at startup; when one changes, the shared Settings footer reveals a *Restart* button. That button now behaves like an alert:

- **Slow red pulse** — while enabled the Restart button animates between bright and dark red (custom template so the theme chrome doesn't paint over it), driven by its `IsEnabled` state so it keeps pulsing across inner-tab switches until the app is restarted.
- **Restart-on-leave prompt** — leaving the Settings area with a restart still pending asks "Restart ioSender now?" (Yes relauches). Switching between the inner Settings tabs does not prompt.
- **Shared restart path** — the relaunch logic is factored into one `DoRestart()` used by both the button and the prompt.

In the integration build. Builds clean (Release, EXIT 0). Applies to every restart-only setting, since all route through the one shared Restart button. Hardware-confirmed ("works great").

#49 Lathe wizards rebuilt to the app layout idiom (high-DPI)

CNC Controls	Lathe/CNC Controls	Lathe/ThreadingWizard.xaml	275	235
CNC Controls	Lathe/CNC Controls	Lathe/TurningWizard.xaml	50	42
CNC Controls	Lathe/CNC Controls	Lathe/FacingWizard.xaml	49	41
CNC Controls	Lathe/CNC Controls	Lathe/PartingWizard.xaml	47	39
CNC Controls	Lathe/CNC Controls	Lathe/ProfileDialog.xaml	40	30

The five lathe wizards were the last upstream code still laid out on an absolute-Margin canvas — fixed pixel Width/Height and hard-coded positions — so their text clipped and controls overlapped once Windows text scaling was set to 125–150% (they were deliberately skipped in the #46 attribute scrub because they needed a real re-layout, not a mechanical edit). Each wizard body was rebuilt with the app’s normal patterns so it flows and scales:

- **Standard idiom** — vertical StackPanel/Grid with Auto sizing, NumericField + ColonAt label alignment, MinWidth/MinHeight floors, a ScrollViewer for overflow, and grouped GroupBox sections.
- **Threading** — a two-column responsive grid (selection / Dimensions / Tool on the left; Thread values / Options / Cut on the right), with the linuxCNC and Mach3 option boxes sharing one slot to preserve the code-behind’s draw-over behaviour, and the Tool schematics + shape radios re-hosted without absolute margins.
- **Turning / Parting / Facing** — a single scrolling input column with Calculate and the generated g-code pinned at the bottom.
- **Profile dialog** — the fixed-size modal became a SizeToContent window; the CSS control and fixed-RPM field share one slot, and it follows the app theme instead of a hard-coded grey.

In the integration build. Builds clean (Release, EXIT 0). Every x:Name, binding, converter and event handler the code-behinds rely on was preserved — no code-behind changes. The rebuilt layouts are functional and scale correctly; some aesthetic cleanup (spacing/grouping refinement) is still wanted at some point, ideally by someone who runs a lathe.

#50 Tab strips fill their width (StretchTabControl)

CNC Controls/CNC Controls/StretchTabControl.cs	119	0
ioSender XL/ioSender XL/MainWindow.xaml	6	4
CNC Controls Lathe/CNC Controls Lathe/LatheWizardsView.xaml	4	3
CNC Controls/CNC Controls/GrblConfigView.xaml	2	2
CNC Controls/CNC Controls/MachineSetupWizard.xaml	2	2
CNC Controls Probing/CNC Controls Probing/ProbingView.xaml	2	2
CNC Controls/CNC Controls/ToolsView.xaml	1	1
CNC Controls/CNC Controls/CNC Controls.csproj	1	0

The tab headers used to sit bunched at the left of each strip with dead space to the right, so tabs ran together and were hard to pick out. A new `StretchTabControl` : `TabControl` spreads them to fill the strip: each tab keeps its natural width (plus one space character each side) scaled up proportionally, so a wider label gets a wider tab and the row fills the width.

- **Assigns each header a width** — the default `TabControl` template hosts its headers in a fixed `TabPanel` (`IsItemsHost`), so a replacement `ItemsPanel` is ignored; setting each `TabItem.Width` is honoured instead.
- **Recomputed off-layout** — widths are recalculated only on the discrete tab add/remove event and, on resize, from a deferred (`DispatcherPriority.Background`) callback that runs *after* the layout pass. Doing it synchronously from inside layout (`SizeChanged/LayoutUpdated`) re-enters the layout system and mis-measures content; the deferred approach sets the widths and lets one ordinary follow-up pass consume them, with no feedback loop. The trade is that during a drag-resize the tabs show old widths for a frame and snap on release.
- **Applied to** the main tab bar and the Settings, Tools, Lathe, Probing and Machine Setup sub-tab strips.

In the integration build. Builds clean (Release, EXIT 0). Confirmed on the running app: strips fill, first tab flush, resize re-spreads cleanly, and tab content still fills/left-aligns (the label-centering was pulled back out after it leaked into content alignment on content-sized views).

#51 SD Card first-line cleanup (overlap + free-space banner)

CNC Controls/CNC Controls/SDCardView.xaml	6	2
CNC Controls/CNC Controls/GrblFilesystems.cs	3	2

Two small fixes to the SD Card tab's top line.

- **No more overlap** — the "List CNC files only" checkbox and the filesystem-status text were absolutely positioned at fixed x offsets, so the status ran under the checkbox label at larger fonts. They now flow in a horizontal `StackPanel`.
- **Banner hides when uninformative** — `FreeSpaceSummary` only lists a filesystem that actually reports free space; the bare "*name*: mounted" fallback carried no information (the file list already shows the filesystem is present) and just added clutter, so the whole banner disappears when the controller reports no sizes.

In the integration build. Builds clean (Release, EXIT 0). Confirmed on the running app.

#52 Per-tool guidance text on the Tools tabs

File	+	-
CNC Controls/CNC Controls/ToolView.xaml	3	0
CNC Controls/CNC Controls/SurfaceSpoilboardWizard.xaml	11	5
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	7	1
CNC Controls/CNC Controls/StepperCalibrationWizard.xaml	8	2
CNC Controls/CNC Controls/AutoSquareWizard.xaml	4	4
CNC Controls/CNC Controls/TrinamicView.xaml	8	2
CNC Controls/CNC Controls/PIDLogView.xaml	5	0

Most of the Tools tabs showed a form with no explanation of what the tool did or how to run it. Each now carries a short “About...” heading + wrapped explainer in the tab’s right-hand pane (matching Auto Square and Stepper Calibration, which already had step text).

- **Covered** — Tool table, Surface spoilboard, Stepper calibration (scratch), Trinamic tuner, PID tuner.
- **Layout tidy** — the Stepper-calibration collision warning is contained to the right column (instructions box fills the height); the Surface-spoilboard result sits in its own light panel distinct from the description; Squareness puts the description above the diagram.
- **Localizable** — every string carries an `x:Uid` so it feeds the locale CSVs; English inline is the fallback.

In the integration build. Builds clean (Release, EXIT 0). Confirmed on the running app.

#53 Headless build script + crash logging

File	+	-
build.ps1	105	0
.vscode/tasks.json	69	0
ioSender XL/ioSender XL/App.xaml.cs	72	7
CLAUDE.md	9	1

Build and run the WPF app entirely from the command line / VS Code, without opening the Visual Studio GUI. (PlatformIO can't drive a .NET Framework build — it only builds the C/C++ firmware/sim — so a plain MSBuild wrapper is the right tool.)

- **build.ps1** — kills a running `ioSender.exe`, builds via MSBuild (located with `vswhere`, Enterprise-path fallback), optionally launches. `-Configuration Debug|Release|Both`, `-Launch`, `-Headless`, `-NoKill`.
- **VS Code tasks** — *Build Debug + Launch* (Ctrl+Shift+B), *Build Release*, *Build Both*, all delegating to the one script.
- **Crash logging** — the global handlers (Dispatcher / AppDomain / TaskScheduler) now dump a timestamped, versioned, full-stack entry to `%AppData%\ioSender\ioSender.crash.log` and exit `0xFA11` (64017). Fixes a real bug where a background-thread exception showed a box but never exited, wedging the process. Interactive runs still show a dialog; `IOSENDER_HEADLESS=1` suppresses it so an unattended crash can't hang on an un-clicked box. VS's JIT debugger does not auto-attach.

In the integration build. Debug + Release build clean; both crash paths verified live (log written, exit `0xFA11`).

#54 Rename LoadStock → Start Job (code matches the UI)

File	+	-
ioSender XL/ioSender XL/StartJobView.xaml(.cs) (was LoadStockView)	9	9
CNC Controls Probing/.../StartJobControl.xaml(.cs) (was LoadStockControl)	9	9
CNC Controls/CNC Controls/StartJobConfig.cs (was LoadStockConfig)	10	10
CNC Controls/CNC Controls/AppConfig.cs	6	6
CNC Controls/CNC Controls/LayoutModel.cs	3	3
CNC Controls/CNC Controls/{ICNCView,LibStrings,StreamPump}	4	4
ioSender XL/.../{MainWindow,JobView}.xaml.cs + IProbeTab.cs	3	3
Locale/*/csv (7 locales, 2 files each)	126	126
*.csproj (3 projects)	7	7

The feature is “Start Job” everywhere in the UI, but the code was still LoadStock*. Full rename — there is no back-compat to preserve, so no migration code.

- **Files & classes** — LoadStockView / Control / Config / Settings → StartJob*.
- **Tokens** — ViewType.LoadStock and ProbingType.LoadStock → StartJob; LayoutKeys value, the folded App.config section (LoadStock / LoadStock.xml), the str_tabLoadStock resource key and its locale-CSV baml paths, and the StreamPump temp-log name all follow.
- **Persisted config** — the three tokens saved in %AppData%\ioSender\App.config (tab name, layout component, section key) were patched out-of-band to match.

In the integration build. Debug + Release build clean; headless startup smoke test passes on the renamed config.

#55 Localization sweep — V2 strings into all 7 locales

File	+	-
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	1300	0
Locale/*/csv/CNC.Controls.Probing.resources.*.csv (7 locales)	147	0
Locale/*/csv/ioSender.resources.*.csv (7 locales)	63	0
Locale/*/csv/CNC.Controls.Viewer.resources.*.csv (7 locales)	35	0
tools/locadd.py	36	1

Later V2 features were x:UId'd in XAML but never had LocBaml rows added, so they fell back to English in the other six languages. The self-deriving `tools/locadd.py` was pointed at the current view set and backfilled every missing row.

- **+1545 rows across 7 locales** — 159 new keys in en-US plus the backfills for keys that had reached en-US but not the other locales, so all seven are consistent again.
- **Covers** — Start Job (renamed view + the probing control), the probing-redesign dialogs (`ProbeMotionParams` / `ProbeDefinition*`), the main-page editor, console search, the Tools tab guidance (#52), the carve view, and assorted flyouts/config controls.
- **Baseline, not machine-translation** — each new row carries the English text for a translator to replace; the app already rendered English via fallback, so nothing changes on screen. Comma values RFC4180-quoted; re-running the tool adds nothing.

In the integration build. Lathe wizards are excluded (no `CNC.Controls.Lathe` locale CSV exists — never LocBaml-localized).